

SAKAR ROBOTICS

Sakar Robot Management Platform

Master Requirements & Technical Specification — Security-First Edition

Version 2.0

Date: 2026-08-26

Classification: Internal — Product & Engineering & Security

Status: Requirements / Design Document — No software implemented, no existing source code modified

Table of Contents

- Document Control
- Part 1 — Cover & Scope Note
- Part 2 — (reserved — see Document Control)
- Part 3 — Executive Summary
- Part 4 — Product Vision
- Part 5 — Business Requirements
- Part 6 — System Architecture
- Part 7 — Applications
- Part 8 — Sakar Cloud
- Part 9 — Data Ownership
- Part 10 — Keenon Cloud vs Sakar Cloud
- Part 11 — Remote Lock/Unlock
- Part 12 — Sakar Robot Event & Log System (SRELS)
- Part 13 — Database
- Part 14 — APIs
- Part 15 — Robot Communication
- Part 16 — Security Architecture
- Part 17 — Security Threat Model
- Part 18 — Security Risk Register
- Part 19 — Authentication & RBAC
- Part 20 — Robot Command Security
- Part 21 — Android / SakarC40Agent Security
- Part 22 — Network Security
- Part 23 — MQTT Security
- Part 24 — WebSocket Security
- Part 25 — Data Protection
- Part 26 — Secrets Management
- Part 27 — Backup & Disaster Recovery
- Part 28 — Monitoring

- Part 29 — Security Testing
- Part 30 — Non-Functional Requirements
- Part 31 — UI Requirements
- Part 32 — Tech Stack
- Part 33 — Development Roadmap
- Part 34 — Acceptance Criteria
- Part 35 — Security Acceptance Gates
- Part 36 — Risks
- Part 37 — Open Questions
- Part 38 — Physical C40 Validation Plan
- Part 39 — Final Recommendation

Document Control

Field	Value
Document title	Sakar Robot Management Platform — Master Requirements & Technical Specification (Security-First Edition)
Version	2.0
Date	2026-08-26
Owner	Sakar Robotics — Product, Engineering & Security
Status	Draft for review — requirements/design only, nothing implemented
Initial target robot	Keenon C40 / C40 S
Physical C40 tested during authoring	NO
Physical Keenon network traffic captured during authoring	NO
Existing source code modified during authoring	NO (SakarC40Agent, Peanut SDK, peanut-sdk-release.aar, backend/frontend/mobile/Android source, database schema, MQTT/WebSocket implementation, Docker deployment config, API implementation — none touched)
Software installed during authoring	NO
C40 connected during authoring	NO

Change summary since v1.0: this revision adds a complete, production-grade security architecture and security hardening specification (Parts 16–29, 35, 38) at the explicit request of a security-architecture review. No requirement from v1.0 was removed; the platform's 20-part structure is expanded to 39 parts. Where the security review's role/permission model (Part 19) refines the provisional role list used in earlier drafts of Part 7 (Applications), Part 19 is the authoritative source going forward — a mapping note is given at the top of Part 19.

Two distinct grading vocabularies are used throughout this document — do not conflate them:

- Technical/robot-capability grading** (used for claims about what the C40, the Peanut SDK, or Keenon Cloud actually do): **CONFIRMED** (proven by a live API test, decompiled bytecode, or direct source read), **LIKELY** (strongly implied, not directly proven), **UNKNOWN** (not established either way), **REQUIRES PHYSICAL C40 TEST** (cannot be resolved

without hardware access), `REQUIRES_NETWORK_TEST` (cannot be resolved without a physical network packet capture), `REQUIRES_VENDOR_SUPPORT` (cannot be resolved without Keenon providing information Sakar does not have access to). No claim in this document states that the C40 physically supports a capability merely because the Peanut SDK exposes an API for it.

2. **Security control status** (used for every security control specified in Parts 16–29 and 35): `PLANNED`, `REQUIREMENT`, `DESIGN_COMPLETE`, `IMPLEMENTATION_PENDING`, `REQUIRES_PHYSICAL_C40_TEST`, `REQUIRES_NETWORK_TEST`, `REQUIRES_VENDOR_SUPPORT`, `UNKNOWN`. The status `IMPLEMENTED` is deliberately never used anywhere in this document, because this is a documentation-only exercise — nothing described here has been built.

Primary source material (read and incorporated before writing): SakarC40Agent source tree (read-only); `peanut-sdk-v1.3.0` including the decompiled `peanut-sdk-release.aar` (read-only); `PEANUT_SDK_C40_TECHNICAL_STUDY.md`; `PEANUT_SDK_C40_API_MATRIX.md`; `KEENON_C40_CLOUD_API_AUDIT.md`; `KEENON_C40_API_TEST_RESULTS.json`; `Peanut SDK v1.3.0 English.docx`; `Open Platform Document V2.4.pdf`; the v1.0 edition of this master document. A project-wide search for any existing "KRLog" documentation, screenshots, or notes returned **no results** — Part 12 (SRELS) is therefore a Sakar-owned design from first principles, not a port of any examined Keenon specification. No file in SakarC40Agent, `peanut-sdk-v1.3.0`, the backend/frontend/mobile projects, the database schema, or any Docker/CI configuration was modified while producing this revision; only this document, its companion documents, and the PDF-generation script were touched.

Part 3 — Executive Summary

What Sakar is building. A Sakar-owned Robot Management Platform — a web application, a mobile application, a robot-resident Android agent (evolving the existing SakarC40Agent), and a Sakar Cloud backend — engineered from the outset as a **security-first** system, so that Sakar, not the robot vendor, is the primary system of record for robot data, control, and access, and so that the platform's highest-risk capability (remotely disabling a robot's motors) is gated on physical proof before it is ever represented as production-ready. The initial target robot is the Keenon C40/C40 S; the architecture is model-agnostic so additional robots can be onboarded later.

Why security-first, specifically. This platform's core value proposition is control over a physical machine that operates around people. A security defect here is not a data breach in the abstract — it is a pathway to an unauthorized person moving, unlocking, or otherwise commanding a robot, or to a customer's fleet data leaking across tenants. Every part of this document from Part 16 onward exists because the product cannot be trusted to ship without them being answered, not because a checklist demanded it.

Why Sakar needs its own platform. The existing Keenon Cloud REST API (independently audited in `KEENON_C40_CLOUD_API_AUDIT.md`) gives Sakar visibility into fleet data Keenon chooses to expose, on Keenon's schema, at Keenon's availability, under Keenon's rate limits, on Keenon's infrastructure. That is workable as a secondary source but not a foundation for a differentiated product, a customer SLA, or a security posture Sakar controls end-to-end.

Why Sakar should own robot operational data. Battery/task/cleaning/error/lock history is the raw material of fleet analytics, customer reporting, support diagnostics, and future billing/predictive-maintenance features — and it is also, in aggregate, sensitive customer operational data that Sakar's own security controls (Parts 16–29), not a vendor's, must govern.

Role of the Peanut SDK. Keenon's local, on-robot Android SDK. `PEANUT_SDK_C40_TECHNICAL_STUDY.md` confirmed — by decompiling `peanut-sdk-release.aar` — that every hardcoded network constant in the SDK is a loopback or private-subnet address, and that no Keenon Cloud hostname exists anywhere in its 1830 scanned classes. This is Sakar's local channel to the robot's battery, motor, navigation, health, and lock state. It is explicitly **not** proven to be the *only* channel the robot's Android computer uses — the OTA/update subsystem and the separate stock Keenon application were both flagged as unverified, and Part 10 and Part 22 carry that caveat forward without softening it.

Role of SakarC40Agent. The existing application that is the sole code path in the Sakar codebase importing the Peanut SDK (confirmed by direct source read). This platform extends it — adding telemetry forwarding, authorized command reception, and the security controls in Part 21 — without replacing its architecture.

Role of Sakar Cloud. The backend that owns robot identity, telemetry, command authorization, task/cleaning orchestration, alerting, audit, and every security control in Parts 16–29.

Role of the Web and Mobile Applications. Clients of Sakar Cloud's APIs only. **Neither application ever communicates directly with a robot** — this is a security architecture principle, not just a convenience (Part 16).

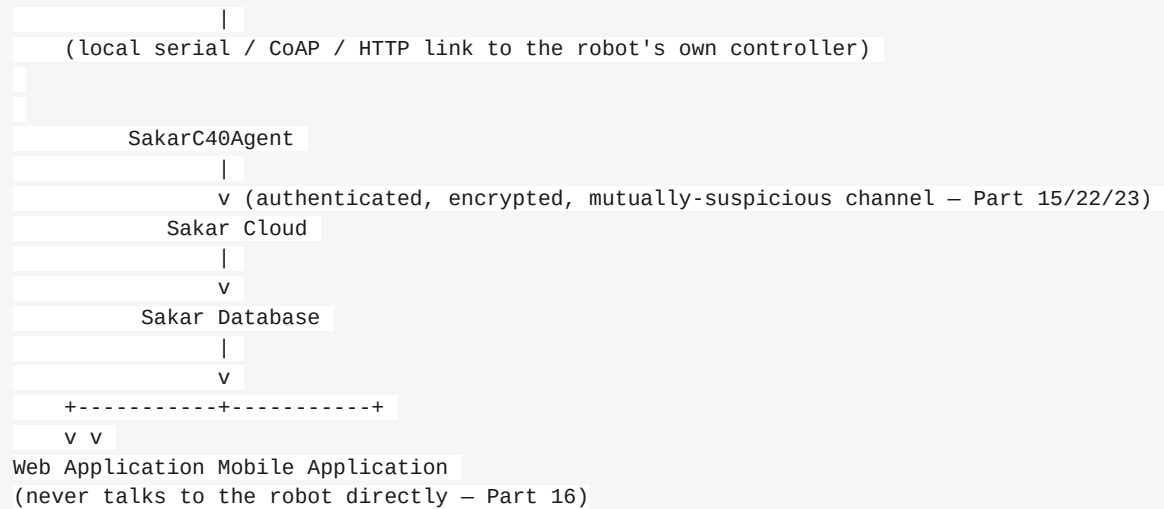
Role of the Robot Logging/Event System. SRELS (Part 12) — a KRLLog-style operational timeline, deliberately separated from the security audit trail because the two have different consumers, different retention needs, and different tamper-resistance requirements.

Remote lock/unlock objective. Give an authorized Sakar user the ability to remotely disable a robot's motors, through an authorization chain at least as strong as: authenticated user -> RBAC -> tenant/site scope -> step-up authentication for unlock -> signed, expiring, replay-protected command -> validated agent -> Peanut SDK. The underlying SDK API (`MotorComponent.enable()`, `MOTOR_ENABLE_LOCK=1/MOTOR_ENABLE_UNLOCK=0`) is **CONFIRMED** to exist. Its **physical effect on a real C40 is not yet confirmed by any source material**, and Part 11/Part 35/Part 38 make it explicit that this feature cannot be marked production-ready until ten specific physical conditions are proven.

Future multi-robot/fleet support. The data model (Part 13) and architecture (Part 6) are built around a `robot_model`s abstraction so a second robot model or vendor can be onboarded without redesigning the platform or its security controls.

Part 4 — Product Vision





Vision statement. Sakar Robotics operates a single robot management platform that any Sakar-deployed robot — starting with the Keenon C40/C40 S — reports into, and that any authorized Sakar or customer user monitors and controls through, with Sakar's own infrastructure as the primary store of record, Sakar's own authorization model as the sole gate on control commands, and Sakar's own security controls as the boundary between "authorized" and "everyone else."

Design principle: the platform must never assume the robot vendor's cloud is reachable, correctly configured, or willing to expose a capability Sakar needs next quarter — and it must never assume a network boundary, an authentication token, or a device is trustworthy without verifying it. Every capability this document specifies is built on what `SakarC40Agent`/Peanut SDK can do *locally*, forwarded to Sakar's own backend, guarded by the controls in Parts 16–29.

Part 5 — Business Requirements

Requirement	Description
Vendor independence	Sakar's core monitoring/control loop must not depend on Keenon Cloud's availability, schema, or willingness to expose a capability
Data ownership	Robot operational data (Part 9) is primarily stored on Sakar infrastructure, governed by Sakar's own retention/access policy

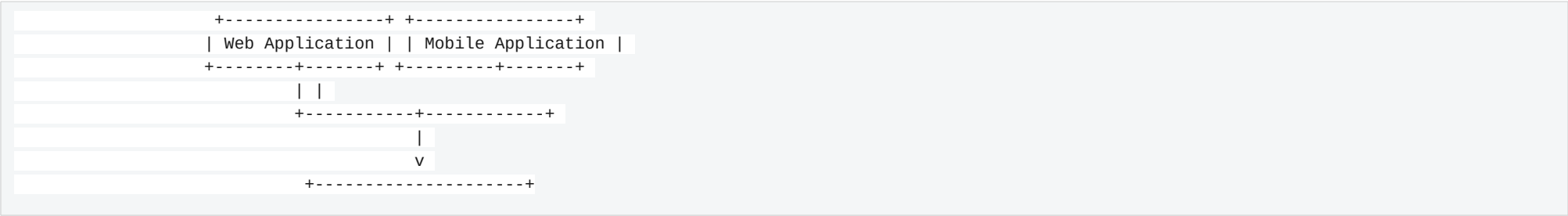
Customer trust	Multi-tenant isolation (Part 19) must be provably strong enough that one customer's data or robots are never reachable by another, even via a crafted request
Safety-first product claims	No customer-facing, marketing, or internal document may claim the remote lock feature works on a physical C40 until Part 38's validation plan is fully executed (Part 11/35)
Extensibility	Onboarding a second robot model/vendor must not require a data-model or security-model redesign — only a new robot_model's row and an agent-side adapter
Security-first posture	Every new feature added after this document is approved must be evaluated against the applicable Part 16–29 controls before it ships, not retrofitted afterward
Compliance readiness	The audit trail (Part 12/13) and data classification (Part 25) are structured so that a future compliance requirement (e.g., a customer security questionnaire, a regulatory audit) can be answered from existing records, not reconstructed after the fact

Stakeholders: Sakar leadership (product direction, commercial risk), Product & Engineering (build), Security (this document's primary author-role), Customer site operators (end users of web/mobile), Support/Technicians (day-to-day operational users), Compliance/Legal (data handling obligations).

Success criteria: (1) the platform's core monitoring/control loop functions with zero required Keenon Cloud calls; (2) no security acceptance gate (Part 35) is bypassed to hit a launch date; (3) the remote lock feature is either physically certified before general availability or explicitly shipped as "best-effort, not guaranteed" with that caveat visible in the product itself, not buried in a document.

Explicit non-goal for this document: implementing any of the above. This is a requirements and architecture specification for review and approval; Part 33 defines the build order that follows approval.

Part 6 — System Architecture





Core architectural principle (repeated because it is load-bearing for the whole security model): the web and mobile applications are clients of Sakar Cloud's API **only**. Neither one ever holds a robot's address, credentials, or a direct connection of any kind. Every robot interaction — read or write — is mediated by Sakar Cloud, which is the single point where authentication, authorization, tenancy, and command-security controls (Parts 19–20) are enforced. This is what makes the security architecture in Part 16 tractable: there is exactly one trust boundary between "the internet" and "a physical robot," not four.

Layered view: presentation (web/mobile) -> application/API (Sakar Cloud) -> robot-facing edge (SakarC40Agent, one instance per robot) -> vendor SDK (Peanut SDK, local-only) -> hardware (C40). Each layer only trusts the layer immediately below it after that layer has proven its identity (Part 19 for users, Part 21 for agents/devices, Part 23 for MQTT clients).

Part 7 — Applications

Note on role names in this Part: the module tables below use the provisional role/permission draft carried forward from v1.0 (SUPER_ADMIN, SAKAR_ADMIN, SAKAR_SUPPORT, CUSTOMER_ADMIN, CUSTOMER_OPERATOR, TECHNICIAN, VIEWER). **Part 19 defines a refined, authoritative RBAC model** (SUPER_ADMIN, ORG_ADMIN, SITE_ADMIN, OPERATOR, TECHNICIAN, VIEWER) produced by this revision's security review. Where the two differ, Part 19 governs; the mapping is approximately CUSTOMER_ADMIN -> ORG_ADMIN, CUSTOMER_OPERATOR -> OPERATOR, SAKAR_ADMIN/SAKAR_SUPPORT -> cross-organization SUPER_ADMIN scope or an org-scoped ORG_ADMIN/SITE_ADMIN grant depending on the

specific Sakar-staff use case, to be finalized at implementation. This module inventory itself (purpose/users/permissions/functions/data/acceptance criteria) is otherwise unchanged from v1.0 and remains accurate.

7.A Web Application

Purpose. Central administration and fleet management surface for Sakar staff and customer administrators.

Login

- **Purpose:** authenticate a user into the platform.
- **Users:** all roles.
- **Permissions:** none required (pre-authentication).
- **Functions:** username/password login, forgot-password flow, MFA challenge where required by role (Part 19).
- **Data:** users, session/refresh tokens.
- **Acceptance criteria:** *Given* valid credentials and a required MFA challenge completed, *when* the user submits login, *then* a session is established and the user lands on the Dashboard scoped to their authorized organizations.

Dashboard

- **Purpose:** fleet-wide operational overview.
- **Users:** all roles (content scoped by permission).
- **Permissions:** ROBOT_VIEW.
- **Functions:** counts of total/online/offline/locked/low-battery/error/cleaning/charging robots; recent alerts; recent audit-relevant events.
- **Data:** robot_status, robot_telemetry, robot_alerts, audit_logs (aggregated).
- **Acceptance criteria:** *Given* a logged-in user, *when* they open the Dashboard, *then* every count reflects only robots under organizations/sites they are authorized for.

Organization Management

- **Purpose:** create/edit/deactivate customer organizations (and the internal Sakar fleet as an organization).
- **Users:** SUPER_ADMIN, SAKAR_ADMIN.
- **Permissions:** USER_MANAGE.
- **Functions:** CRUD on organizations; assign an org admin to an organization.
- **Data:** organizations.

- **Acceptance criteria:** *Given* a `SAKAR_ADMIN`, *when* they create an organization with a unique name, *then* it appears in the organization list and can subsequently have sites/robots assigned.

Site Management

- **Purpose:** manage physical locations (sites) under an organization.
- **Users:** `SUPER_ADMIN`, `SAKAR_ADMIN`, `CUSTOMER_ADMIN` (own org only).
- **Permissions:** `USER_MANAGE`/org-scoped equivalent.
- **Functions:** CRUD on `sites`; assign robots to a site.
- **Data:** `sites`.
- **Acceptance criteria:** *Given* a `CUSTOMER_ADMIN`, *when* they create a site under their own organization, *then* it is visible only to users authorized for that organization.

Robot Management

- **Purpose:** register, activate/deactivate, and configure robots.
- **Users:** `SUPER_ADMIN`, `SAKAR_ADMIN`, `SAKAR_SUPPORT`, `CUSTOMER_ADMIN` (own org).
- **Permissions:** `ROBOT_VIEW` (read), `ROBOT_CONFIGURE` (write).
- **Functions:** add robot (identity: `serial/mftCode` — `CONFIRMED` available from Keenon Cloud's robot list today), activate/deactivate, view/edit configuration.
- **Data:** `robots`, `robot_models`, `robot_credentials`.
- **Acceptance criteria:** *Given* an authorized admin, *when* a valid robot identity is registered, *then* the robot appears in the fleet list scoped to the correct organization/site.

Fleet Management

- **Purpose:** group robots (by site, function, or arbitrary tag) for bulk viewing/reporting.
- **Users:** `SUPER_ADMIN`, `SAKAR_ADMIN`, `CUSTOMER_ADMIN`, `CUSTOMER_OPERATOR`.
- **Permissions:** `ROBOT_VIEW`.
- **Functions:** create/manage robot groups; assign robots to groups.
- **Data:** `robots` (group/tag association).
- **Acceptance criteria:** *Given* a robot group, *when* a user with `ROBOT_VIEW` opens it, *then* they see only member robots they are individually authorized for.

Robot Live Monitoring

- **Purpose:** real-time (or near-real-time) view of a robot's current state.
- **Users:** all roles with `ROBOT_VIEW`.

- **Permissions:** `ROBOT_VIEW`.
- **Functions:** display current battery, charging state, motor status, navigation status, health, lock state, connection state.
- **Data:** `robot_status`, `robot_telemetry` (latest values).
- **Acceptance criteria:** *Given* an online robot with a functioning agent, *when* new telemetry arrives, *then* the live view updates within the configured freshness window (Part 30).

Robot Details

- **Purpose:** deep-dive single-robot view.
- **Users:** all roles with `ROBOT_VIEW`.
- **Permissions:** `ROBOT_VIEW`.
- **Functions:** identity, firmware/app version, battery/charging history, motor/health, odometer, last communication, event/error timeline, logs.
- **Data:** `robots`, `robot_telemetry`, `robot_events`, `robot_errors`.
- **Acceptance criteria:** *Given* a registered robot, *when* its details page is opened, *then* every field displays either a real value or an explicit "unavailable" state — never a fabricated placeholder.

Maps

- **Purpose:** visualize robot position, cleaning path, target points, charging stations.
- **Users:** all roles with `ROBOT_VIEW`; `ROBOT_CONFIGURE` for editing.
- **Permissions:** `ROBOT_VIEW` (view), `ROBOT_CONFIGURE` (manage).
- **Functions:** static floor-plan display (`CONFIRMED`); named target points with coordinates (`CONFIRMED`); live position overlay (`REQUIRES PHYSICAL C40 TEST`); cleaning-coverage overlay via snapshot images (`CONFIRMED`); restricted zones (`UNKNOWN` — **P2/future**, pending vendor confirmation).
- **Data:** `maps`, `map_points`.
- **Acceptance criteria:** *Given* a robot with a known scene/floor, *when* the map view is opened, *then* the floor plan and named points render; live position rendering is explicitly out of MVP acceptance until physically validated.

Tasks

- **Purpose:** create and track delivery/utility tasks (as applicable to the robot model).
- **Users:** roles with `ROBOT_TASK_CREATE`/`ROBOT_TASK_CANCEL`.
- **Permissions:** `ROBOT_TASK_CREATE`, `ROBOT_TASK_CANCEL`.
- **Functions:** create task, assign robot, start/pause/resume/stop/cancel, view history/status.
- **Data:** `robot_tasks`, `task_events`.

- **Acceptance criteria:** *Given* an authorized operator, *when* a task is created and assigned to an available robot, *then* it transitions through a defined lifecycle recorded in `task_events`.

Cleaning

- **Purpose:** C40-relevant cleaning-specific functionality.
- **Users:** roles with `ROBOT_TASK_CREATE/ROBOT_TASK_CANCEL`.
- **Permissions:** same as Tasks.
- **Functions and confidence:**

Capability	Status
Cleaning history (what ran, when, duration, area, efficiency, failure reason, snapshot image)	CONFIRMED — Keenon Cloud <code>clean/log/list</code> , live-tested, 697 records for the current fleet
Cleaning schedule read	CONFIRMED — Keenon Cloud <code>clean/strategy/list</code> , live-tested, 3 active schedules
Cleaning mode / return-point query	CONFIRMED API exists (Keenon Cloud, documented; not yet live-tested)
Create/edit a cleaning schedule from Sakar	CONFIRMED API exists, classified <code>NON_PHYSICAL_WRITE</code> in the Cloud audit, not yet exercised
Trigger an immediate/temporary cleaning task	CONFIRMED API exists, classified <code>PHYSICAL_ROBOT_CONTROL</code> , not yet exercised — requires physical validation before production use

- **Data:** `cleaning_sessions`.
- **Acceptance criteria:** *Given* an online C40 with cleaning history, *when* the Cleaning module is opened, *then* historical sessions display with duration/area/efficiency; schedule-editing and immediate-task features are labeled "pending physical validation" until confirmed.

Charging

- **Purpose:** view/manage charging state and history.
- **Users:** roles with `ROBOT_VIEW/ROBOT_CONTROL`.
- **Permissions:** `ROBOT_VIEW` (view), `ROBOT_CONTROL` (start/stop charge).
- **Functions:** current charge state, charging history, manual/auto charge start, stop charge.
- **Data:** `charging_sessions`.
- **Acceptance criteria:** *Given* a robot reporting a charge event, *when* it is received, *then* a `charging_sessions` record is created/updated and reflected in the UI.

Alerts

- **Purpose:** surface actionable conditions.
- **Users:** all roles with `ROBOT_VIEW`.
- **Permissions:** `ROBOT_VIEW`.
- **Functions:** error/warning/critical/offline/low-battery/emergency/task-failure alerts, acknowledgement.
- **Data:** `robot_alerts`.
- **Acceptance criteria:** *Given* a telemetry value crossing a configured threshold, *when* it is ingested, *then* an alert is created and visible to authorized users within the configured latency (Part 30).

Errors

- **Purpose:** dedicated error inventory (distinct from generic alerts and from the event timeline).
- **Users:** all roles with `ROBOT_VIEW`.
- **Permissions:** `ROBOT_VIEW`.
- **Functions:** list/filter robot errors by severity, source, resolution state.
- **Data:** `robot_errors`.
- **Acceptance criteria:** *Given* an SDK-reported error/health-check failure, *when* it is ingested, *then* it appears in the Errors module with the fields defined in Part 12.C.

Robot Logs

- **Purpose:** the SRELS timeline UI (Part 12) — the KRLog-style operational history.
- **Users:** all roles with `ROBOT_LOG_VIEW`; `AUDIT_VIEW` for the security-log overlay.
- **Permissions:** `ROBOT_LOG_VIEW`, `AUDIT_VIEW`.
- **Functions:** filterable chronological timeline (all/info/warning/error/critical/security/commands/navigation/battery/charging/SDK/system).
- **Data:** `robot_events`, `robot_errors`, `robot_commands`, `application_logs`, `audit_logs` (security-scoped view).
- **Acceptance criteria:** see Part 12.F.

Robot Events

- **Purpose:** structured event browser (a filtered slice of Robot Logs, event-only).
- **Users:** all roles with `ROBOT_VIEW`.
- **Permissions:** `ROBOT_VIEW`.
- **Functions:** browse `robot_events` by type/severity/time range.
- **Data:** `robot_events`.
- **Acceptance criteria:** *Given* a defined event type (Part 12.B), *when* it occurs, *then* it is queryable in this module within the ingestion latency window.

Analytics

- **Purpose:** aggregate reporting.
- **Users:** roles with `AUDIT_VIEW` or a future dedicated analytics permission.
- **Permissions:** `AUDIT_VIEW` (baseline; refine as noted).
- **Functions:** utilization, cleaning duration, battery usage, charging duration, task completion rate, error frequency, downtime, online/offline history, robot performance.
- **Data:** derived from `robot_telemetry`, `robot_events`, `robot_tasks`, `cleaning_sessions`, `charging_sessions`.
- **Acceptance criteria:** *Given* at least one full day of telemetry for a robot, *when* the Analytics module is opened for it, *then* utilization/uptime figures reconcile against the raw event history for the same window.

Users / Roles / Permissions

- **Purpose:** RBAC administration.
- **Users:** `SUPER_ADMIN`, `SAKAR_ADMIN`, `CUSTOMER_ADMIN` (own org).
- **Permissions:** `USER_MANAGE`, `ROLE_MANAGE`.
- **Functions:** CRUD users, assign roles, view the permission matrix (Part 19).
- **Data:** users, roles, permissions.
- **Acceptance criteria:** *Given* a `CUSTOMER_ADMIN`, *when* they create a user within their own organization, *then* that user cannot be assigned a role/permission scope outside that organization.

Audit Logs

- **Purpose:** immutable security/compliance trail.
- **Users:** roles with `AUDIT_VIEW`.
- **Permissions:** `AUDIT_VIEW`.
- **Functions:** query by user/org/robot/action/time range; export.
- **Data:** `audit_logs`.
- **Acceptance criteria:** see Part 34.

Robot Lock/Unlock (dedicated module)

- **Purpose:** the platform's single most safety-critical feature. Full treatment in Part 11.
- **Users:** roles with `ROBOT_LOCK`/`ROBOT_UNLOCK`.
- **Permissions:** `ROBOT_LOCK` (weaker grant), `ROBOT_UNLOCK` (strictly stronger grant, step-up authentication required — Part 19).

- **Functions:** lock, unlock, view current lock state/reason/locked-by/timestamp, view lock history.
- **Data:** robot_locks.
- **Acceptance criteria:** see Part 11 and Part 34 — **the physical-effect criterion cannot be marked PASS until Part 38 is completed.**

Settings

- **Purpose:** platform-level configuration (alert thresholds, notification preferences, retention windows within policy limits).
- **Users:** SUPER_ADMIN, SAKAR_ADMIN.
- **Permissions:** SYSTEM_ADMIN.
- **Functions:** configure thresholds and preferences per organization.
- **Data:** organization-scoped settings.
- **Acceptance criteria:** *Given* an admin changes an alert threshold, *when* subsequent telemetry crosses the new threshold, *then* the alert fires according to the updated value, not the old one.

7.B Mobile Application

Target: Android and iOS. **Recommended technology:** Flutter, unless an existing Sakar mobile standard supersedes this (none was found in the reviewed materials).

Feature	Priority	Notes
Login	P0	Same auth backend as web
Dashboard	P0	Fleet summary, scoped by authorization
Robot list	P0	Name, ID, model, serial, online/offline, battery, state, lock status, current task
Robot details	P0	Same field set as web Robot Details
Live robot status	P0	Battery, charging, motor, navigation, health
Robot location	P1	REQUIRES PHYSICAL C40 TEST — display only once real data exists
Cleaning state	P1	CONFIRMED data source (Keenon Cloud, live-tested)
Tasks	P1	View + control per permission

Alerts	P0	Push notifications, see below
Errors	P0	Read-only
Notifications	P0	Robot offline, low battery, critical error, robot locked/unlocked, task completed/failed, charging started/completed, emergency event, communication lost
Lock	P0 (architecture), P1 (physical certification)	Per Part 11
Unlock	P0 (architecture), P1 (physical certification)	Requires ROBOT_UNLOCK + step-up authentication
Robot history	P1	Filtered SRELS timeline, mobile-optimized
User profile	P0	Session/device management, MFA setup

Role-based access: identical RBAC model as web (Part 19) — the mobile app is a client of the same authorization service, not a separately-privileged surface.

7.C Robot Android Tablet Application — "SakarC40Agent"

Runs directly on the C40's onboard Android computer. This is the existing project, extended.

Responsibility	Status
Peanut SDK initialization	CONFIRMED — already implemented (<code>PeanutSdkBridge.init</code>)
Robot communication (local link)	CONFIRMED mechanism exists; exact <code>LinkType</code> for the real C40 is UNKNOWN
Telemetry collection	CONFIRMED for battery, motor status/health, runtime state, work mode, odometer, robot IP, robot properties, emergency state; navigation status and position are CONFIRMED as APIs but REQUIRES PHYSICAL C40 TEST for real values
Robot state	CONFIRMED — <code>RuntimeInfo</code>
Health / errors / events	CONFIRMED API surface (65 named topics); most payload schemas beyond the documented 16 are UNKNOWN
Charging status	CONFIRMED

Lock/unlock where physically supported	SDK API CONFIRMED; physical support REQUIRES PHYSICAL C40 TEST — see Part 11
Command execution	New capability to be added, governed by Part 20
Communication with Sakar Cloud	New capability to be added, governed by Part 15/22/23
Local safety behavior	Must not depend on Sakar Cloud reachability (Part 21/30)
Offline behavior	See Part 30
Agent health / version / device identity	New capability to be added — required for fleet-wide agent management and for Part 21's device-identity control

Explicit constraint: do not invent unsupported C40 functionality. Every row above is graded per the source studies; none claims physical behavior beyond what has been proven.

Part 8 — Sakar Cloud

Recommended technology stack and rationale:

Technology	Role	Why
Java + Spring Boot	Backend application framework	Mature, strongly-typed, first-class support for the security (Spring Security), scheduling, and REST tooling this platform needs
Spring Security	AuthN/AuthZ framework	Directly supports the JWT + RBAC model in Part 19 without building auth primitives from scratch
PostgreSQL	Primary database	Relational integrity for the strongly-relational fleet/org/user model (Part 13)
Redis	Cache / ephemeral state	Session/token caching, rate-limiting counters, hot-path "latest telemetry" reads

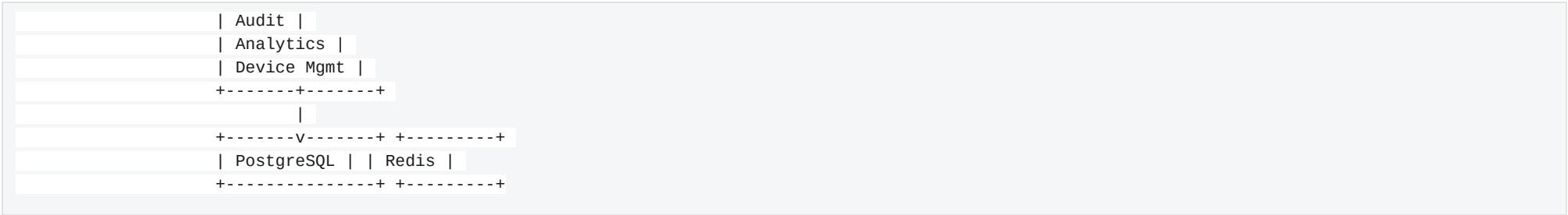
MQTT	Robot-facing real-time transport	Purpose-built for constrained, intermittently-connected devices (Part 15/23)
WebSocket	Web/mobile-facing real-time transport	Live dashboard push (Part 15/24)
REST API	Primary application-facing API	Simple, cacheable, universally supported (Part 14)
Docker	Packaging/deployment	Consistent environments across dev/staging/production
Linux	Host OS	Standard for the above stack
Nginx	Reverse proxy / TLS termination	TLS termination, routing, static asset serving in front of the backend

Backend modules (logical services): Authentication, Authorization, Organization, Site, Robot Registry, Robot Telemetry, Robot Commands, Task Management, Cleaning Management, Map Management, Alert Management, Notification, SRELS, Robot Events, Audit, Analytics, Device Management.

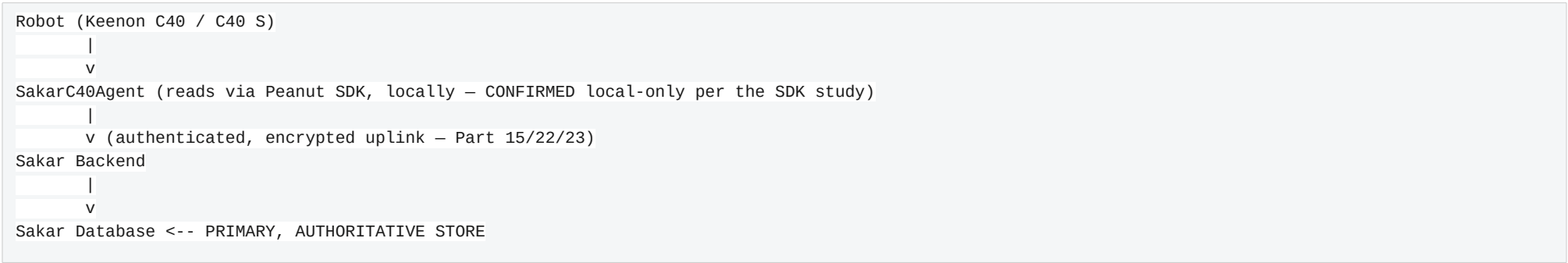
On microservices: do not decompose into microservices unless a specific module demonstrably needs independent scaling. The practical initial architecture is a single well-modularized Spring Boot application (a "modular monolith") with module boundaries enforced in code. The Telemetry ingestion path is the most likely first candidate for independent scaling.

Security placement note: the diagram below shows the *logical* module layout. Part 16 shows the same backend placed behind a WAF/firewall inside a segmented private network — the two diagrams describe the same system at different levels of detail.





Part 9 — Data Ownership



Data categories and where they are proven to originate:

Category	Confirmed available locally (SDK)	Available via Keenon Cloud	Requires physical C40 validation	Unknown
Robot identity	CONFIRMED (DeviceComponent.getBoardInfo)	CONFIRMED (live-tested: robotId, robotCode, mftCode, robotModel)	—	—
Battery	CONFIRMED	CONFIRMED (live-tested)	—	—
Charging	CONFIRMED	CONFIRMED (schedules live-tested; real-time charge state not populated for current fleet)	—	—

Runtime (workMode/syncStatus/odo)	CONFIRMED	Partial (appVersion live-tested)	—	—
Motor	CONFIRMED (status/health)	Not exposed by Keenon Cloud at all	Physical lock effect — see Part 11	—
Navigation	CONFIRMED API	Not confirmed for C40 in the Cloud audit	Real navigation-state values	—
Position	CONFIRMED API exists	CONFIRMED empty for current fleet (data: null, live-tested)	Real coordinate values	—
Tasks	—	Cloud-side task-record schema exists but is empty for this account's business type	—	Sakar-side task orchestration is a new, Sakar-owned concept regardless
Cleaning	Partial (topic names only, payload UNKNOWN)	CONFIRMED — 697 historical records, live-tested	—	Full SDK-side cleaning payload schema
Errors/Alarms	CONFIRMED API	CONFIRMED (globalState.faulting, live-tested)	—	Most of the 65 SDK topics' payload schemas
Health	CONFIRMED API	CONFIRMED (cleaning-family health fields, live-tested)	—	—
Events	CONFIRMED topic names (65)	CONFIRMED webhook callback types documented	—	Payload schema for ~49 of 65 topics
Maps	CONFIRMED API (MapComponent)	CONFIRMED (real PNG + points, live-tested)	Live position overlay	—
Odometer	CONFIRMED	Not directly exposed by Cloud for C40	—	—
Robot history	New Sakar-owned concept (SRELS, Part 12)	Partial (cleaning logs, live-tested)	—	—
Logs	New Sakar-owned concept	—	—	—

Data lifecycle requirements (expanded in Part 25/27/30): what is collected — everything above graded **CONFIRMED**; where stored — Sakar's PostgreSQL as primary, with defined hot/cold retention tiers; who can access — governed by Part 19's RBAC/multi-tenant model; encryption — Part 25; backup — Part 27; deletion — supports organization/robot offboarding with a defined data-purge procedure; export — audit-logged, permission-gated.

Part 10 — Keenon Cloud vs Sakar Cloud

Capability	Keenon Cloud	Peanut SDK (local)	Sakar Platform (recommended primary)
Battery	CONFIRMED live	CONFIRMED (bytecode)	Peanut SDK via SakarC40Agent — direct, no cloud round-trip
Online/status	CONFIRMED live	CONFIRMED (heartbeat/runtime)	Peanut SDK for real-time
Position	CONFIRMED empty for current fleet	API exists, untested	Peanut SDK , pending Part 38-style physical validation
Navigation	Not confirmed for C40 in Cloud audit	CONFIRMED API	Peanut SDK , pending physical test
Task/task history	CONFIRMED (cleaning family, 697 records)	No single "task history" API found	Both
Errors/health	CONFIRMED both	CONFIRMED both	Both
Maps	CONFIRMED live (real image)	CONFIRMED API, different mechanism	Keenon Cloud for dashboard imagery today; SDK for on-robot purpose
Charging	CONFIRMED (schedules)	CONFIRMED (real-time)	Peanut SDK for real-time state
Motor	Not exposed at all	CONFIRMED fully	Peanut SDK only
Lock	Not exposed at all	CONFIRMED API exists, physical effect unconfirmed	Peanut SDK only , pending Part 38
Robot identity/firmware	CONFIRMED live	CONFIRMED (bytecode)	Either

What Sakar should own: all operational telemetry, all command authorization and history, all lock/unlock state, all task/cleaning orchestration, all audit and access-control data.

What must not depend on Keenon Cloud: the platform's core monitoring and control loop.

Explicit caveat — network communication has been *studied*, not *proven absent*. The SDK study found no hardcoded Keenon Cloud endpoint in the analyzed SDK transport code — every network constant it found was local/private (127.0.0.1, 192.168.64.20, 192.168.64.10, and local-only ports for CoAP/HTTP/WebSocket/TFTP). This is a **CONFIRMED** finding about the SDK's own hardcoded transport constants. It is **not** the same claim as "the C40 never sends data to Keenon Cloud," and this document does not make that broader claim. Three things remain explicitly open:

Item	Status
Peanut SDK's own hardcoded network transport constants	CONFIRMED local-only (decompiled bytecode)
Peanut SDK's OTA/update subsystem's actual traffic	UNKNOWN — the update-package source was not traced to bytecode depth; REQUIRES NETWORK TEST
Stock Keenon Android application's network behavior	UNKNOWN — a separate APK, not analyzed at all in any source study; REQUIRES NETWORK TEST and possibly REQUIRES VENDOR SUPPORT
Any other Android-OS-level or background-service network traffic on the tablet	UNKNOWN; REQUIRES NETWORK TEST

Governing rule for every document derived from this one: "zero Keenon outbound communication" may not be claimed — to a customer, to leadership, or in marketing material — until the physical network test in Part 22/38 has been performed and its results reviewed. This rule is restated in Part 22 and Part 38 and must not be weakened in any summary of this document.

Part 11 — Remote Lock/Unlock

This is the single most safety-critical requirement in the entire platform.

Requirement: an authorized Sakar user must be able to remotely lock a robot's motors, and an authorized Sakar user holding a **stronger** authorization must be able to unlock them — subject in both directions to the physical validation in Part 38.

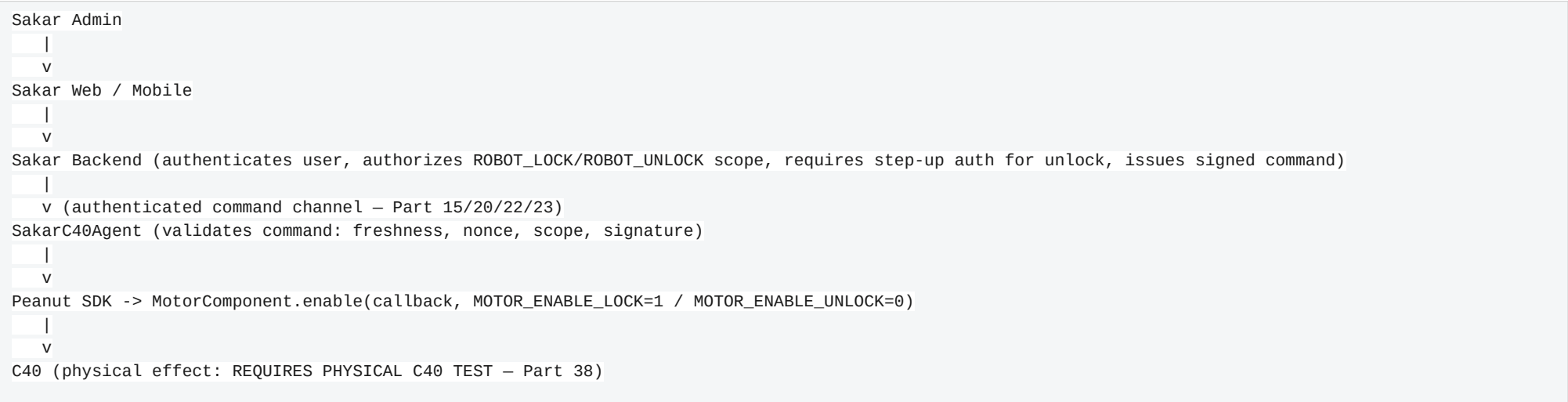
Authorization model:

Action	Permission	Additional requirement
LOCK	ROBOT_LOCK	Reason (free text, required); audit record

UNLOCK	ROBOT_UNLOCK — strictly stronger grant than ROBOT_LOCK (Part 19)	Reason (required); step-up authentication/MFA preferred/required by role (Part 19); audit record
--------	---	---

Audit fields for every lock/unlock action: user, organization, site, robot, command_id, timestamp, ip, device, reason, result.

Flow:



Grading, restated in full because this is the platform's highest-risk feature:

Claim	Status
SDK exposes a lock API	CONFIRMED — <code>MotorComponent.enable(IDataCallback, int)</code> , <code>ApiConstants.MOTOR_ENABLE_LOCK=1</code> , <code>MOTOR_ENABLE_UNLOCK=0</code> — values recovered from the compiled AAR's constant pool; the vendor's own documentation only says "on or off"
SDK reports lock status	CONFIRMED — <code>MotorComponent.getStatus()</code> returns 255 (CODE_LOCKED), 16 (CODE_BUTTON_UNLOCKED), or 32 (CODE_APP_UNLOCKED)
Lock physically prevents robot movement	REQUIRES PHYSICAL C40 TEST
Navigation cannot bypass the lock	REQUIRES PHYSICAL C40 TEST — no cross-reference between <code>NavigationComponent/PeanutNavigation</code> and motor-lock state exists anywhere in the decompiled SDK

Manual control cannot bypass the lock	REQUIRES PHYSICAL C40 TEST — <code>MotorComponent.manual()/forward()/backward()/turnLeft()/turnRight()</code> are separate API calls from <code>enable()</code> ; whether the firmware itself rejects them while locked is unconfirmed
Lock survives agent restart	REQUIRES PHYSICAL C40 TEST
Lock survives robot reboot	REQUIRES PHYSICAL C40 TEST
Lock survives network/link loss	REQUIRES PHYSICAL C40 TEST
Unauthorized local user cannot bypass the lock	Depends on the rows below; cannot be claimed true today
Stock Keenon app cannot bypass the lock, or OS-level enforcement prevents it	LIKELY FALSE as an app-only claim — the SDK's own Integration Guide treats any app holding a valid Peanut SDK license as a peer with equal access to <code>MotorComponent.enable()</code> ; genuinely preventing bypass requires an Android OS/device-management control (Part 21), not application logic alone
Only authorized Sakar users can unlock	CONFIRMED achievable in the <i>software authorization layer</i> Sakar builds (Part 19); does not by itself prevent the device-level bypass path above
Power-cycle behavior is understood	REQUIRES PHYSICAL C40 TEST

Mandatory physical test requirements before this feature may be marked production-ready (full test procedures in Part 38): 1. Lock prevents physical movement. 2. Navigation cannot bypass the lock. 3. Manual control cannot bypass the lock. 4. Lock survives agent restart. 5. Lock survives robot reboot. 6. Lock survives network loss. 7. Unauthorized local user cannot bypass the lock. 8. Stock Keenon application cannot bypass the lock (or OS-level enforcement demonstrably prevents it). 9. Authorized Sakar user can unlock. 10. Power-cycle behavior is understood and documented.

Product requirement (non-negotiable): the platform must ship with an explicit, user-visible maturity flag distinguishing `LOCK API: Planned` (buildable now) from `SDK implementation: Available` (the call is real and callable now) from `Physical C40 behavior: REQUIRES TEST` (not yet proven). Until all ten conditions above are physically validated, the UI/API must present lock as **requested/best-effort**, never as a guaranteed safety control. See Part 35 (Security Acceptance Gates) for the formal go/no-go gate this maps to.

Part 12 — Sakar Robot Event & Log System (SRELS)

Name: Sakar Robot Event & Log System (SRELS). No existing "KRLog" documentation, screenshots, or notes were found anywhere in the reviewed project directories — SRELS is designed here as a Sakar-owned analog to the concept described, not a port of any examined Keenon specification.

Governing principle: this must not be a single console-log stream. Six distinct concerns are deliberately separated, because they have different consumers, different retention needs, and different sensitivity:

- 1. **Application Logs** — agent/backend operational logs (developer/DevOps consumer).
- 2. **Robot Events** — structured, meaningful things that happened to the robot (operator consumer).
- 3. **Robot Errors** — a dedicated error inventory with resolution lifecycle (support/QA consumer).
- 4. **Robot Command Log** — full lifecycle of every command sent to a robot (operator + security consumer).
- 5. **Security/Audit Logs** — compliance-grade, immutable trail of sensitive actions (security/compliance consumer).
- 6. **Telemetry** — raw time-series values (analytics consumer) — specified in Part 9/13.

Structured-logging requirement (applies to every category above): every log record must carry, at minimum: a correlation/request ID, a timestamp, a severity, a source, and — where applicable — robot_id, organization_id, site_id, user_id, and command_id. This is what makes it possible to reconstruct "everything that happened around event X" across all six categories by joining on shared identifiers, and it is what makes the SRELS timeline (Part 12.F) possible at all.

Immutability requirement: security-sensitive events (category 5, and the security-relevant subset of categories 2 and 4 — e.g., MOTOR_LOCKED/MOTOR_UNLOCKED, every row in the command log) must be append-only from the application's perspective — no UI, no API, and no ordinary database role may update or delete them. See Part 13 for the database-level enforcement of this.

12.A Application Logs

Operational logs from the agent and backend processes themselves — not robot state. Examples: SDK_INITIALIZED, SDK_CONNECTED, SDK_DISCONNECTED, SERVER_CONNECTED, SERVER_DISCONNECTED, HEARTBEAT, TELEMETRY_RECEIVED. Diagnostic, high-volume, short-retention by design.

12.B Robot Events

Event	Status
ROBOT_ONLINE / ROBOT_OFFLINE	CONFIRMED — derived from heartbeat presence/absence
BATTERY_LOW	CONFIRMED — derived from power/batteryLevel crossing a configured threshold
CHARGING_STARTED / CHARGING_STOPPED	CONFIRMED — BatteryComponent/PeanutCharger events

MOTOR_LOCKED / MOTOR_UNLOCKED	CONFIRMED as a reported-status transition (getStatus() change); physical effect unconfirmed , see Part 11
EMERGENCY_TRIGGERED / EMERGENCY_RELEASED	CONFIRMED — RuntimeInfo.isEmergencyOpen()/topic BUTTON_STATUS
NAVIGATION_STARTED / NAVIGATION_COMPLETED / NAVIGATION_FAILED	CONFIRMED API exists; real-robot occurrence REQUIRES PHYSICAL C40 TEST
TASK_STARTED / TASK_COMPLETED / TASK_FAILED	Sakar-owned concept (task orchestration), built on top of the above

Any event not in this table must not be implemented or advertised until its data source is confirmed and this table is updated.

12.C Robot Errors

Fields: robot_id, error_code, severity, source, message, sdk_api, timestamp, status (open/acknowledged/resolved), resolved_at, resolved_by.

12.D Robot Command Log

Lifecycle: COMMAND_CREATED -> COMMAND_AUTHORIZED -> COMMAND_SENT -> COMMAND_RECEIVED -> SDK_EXECUTED -> COMMAND_SUCCESS | COMMAND_FAILED | COMMAND_TIMEOUT.

Fields: command_id, robot_id, user_id, command, timestamp, status, request (payload), response, error, duration_ms.

12.E Security/Audit Log

Examples: LOGIN, LOGOUT, ROBOT_REGISTERED, ROBOT_LOCKED, ROBOT_UNLOCKED, TASK_CREATED, TASK_CANCELLED, USER_CREATED, ROLE_CHANGED, PERMISSION_CHANGED, ROBOT_CONFIGURATION_CHANGED.

Fields: user, organization, robot, action, result, reason, ip, device, timestamp, request_id.

12.F Robot Timeline

A KRLog-style chronological operational history combining robot_events, robot_errors, robot_commands, and a security-scoped view of audit_logs into one filterable stream per robot.

Example rendering:

```
14:32:10 INFO Robot online
14:32:11 INFO SDK connected
14:33:04 INFO Battery 83%
14:35:22 WARN Navigation blocked
14:35:25 INFO Robot locked
14:35:26 INFO Lock acknowledged
14:40:18 ERROR Navigation failed
```

Dashboard filters: All, Info, Warning, Error, Critical, Security, Commands, Navigation, Battery, Charging, SDK, System.

Acceptance criteria: *Given* a robot with events, errors, and commands recorded within a time window, *when* a user with `ROBOT_LOG_VIEW` opens its timeline and applies a filter, *then* only matching entries render, in strict chronological order, with no cross-robot leakage and no security-log entries visible to a user lacking `AUDIT_VIEW`.

Part 13 — Database

Full field-level detail (types, constraints, example records) lives in the companion document `SAKAR_ROBOT_PLATFORM_DATABASE.md`. Complete entity list:

`organizations`, `sites`, `users`, `roles`, `permissions`, `robots`, `robot_models`, `robot_credentials`, `robot_status`, `robot_telemetry`, `robot_events`, `robot_errors`, `robot_alerts`, `robot_commands`, `command_results`, `robot_locks`, `robot_tasks`, `task_events`, `maps`, `map_points`, `cleaning_sessions`, `charging_sessions`, `notifications`, `audit_logs`, `application_logs`.

Core relationships:

```
organizations 1--* sites 1--* robots *--1 robot_models
robots 1--1 robot_credentials
robots 1--* robot_status (latest-state, upserted)
robots 1--* robot_telemetry / robot_events / robot_errors / robot_alerts
robots 1--* robot_commands 1--1 command_results
robots 1--* robot_locks
robots 1--* robot_tasks 1--* task_events
robots 1--* maps 1--* map_points
robots 1--* cleaning_sessions / charging_sessions
users *--1 organizations, users *--* roles *--* permissions
users 1--* notifications
(organization_id | robot_id) attached to every audit_logs row
```

Cross-cutting requirements for every table: an `organization_id` reachable directly or via `robot_id -> site_id -> organization_id`, enforced at the data-access layer (Part 19); `created_at/updated_at` timestamps; explicit retention policy.

SRELS tables (Part 12), restated with retention detail:

Table	Purpose	Key fields	Retention
<code>robot_events</code>	Structured event history	<code>id</code> , <code>robot_id</code> , <code>event_type</code> , <code>severity</code> , <code>payload</code> (JSONB), <code>occurred_at</code>	Hot 90 days, cold/aggregated thereafter
<code>robot_errors</code>	Error inventory with resolution lifecycle	<code>id</code> , <code>robot_id</code> , <code>error_code</code> , <code>severity</code> , <code>source</code> , <code>message</code> , <code>status</code> , <code>occurred_at</code> , <code>resolved_at</code> , <code>resolved_by</code>	Hot 180 days, then archived
<code>robot_commands</code>	Full command lifecycle	<code>id</code> , <code>robot_id</code> , <code>user_id</code> , <code>command_type</code> , <code>status</code> , <code>request/response</code> (JSONB), <code>created_at</code> , <code>sent_at</code> , <code>completed_at</code> , <code>duration_ms</code>	1 year minimum
<code>audit_logs</code>	Immutable security/compliance trail	<code>id</code> , <code>user_id</code> , <code>organization_id</code> , <code>robot_id</code> , <code>action</code> , <code>result</code> , <code>reason</code> , <code>ip_address</code> , <code>device</code> , <code>request_id</code> , <code>occurred_at</code>	Minimum 1 year, append-only — no update/delete path in the product surface
<code>robot_telemetry</code>	Raw time-series values	<code>id</code> , <code>robot_id</code> , <code>metric</code> , <code>value</code> , <code>recorded_at</code>	Hot 90 days at full resolution, downsampled thereafter
<code>application_logs</code>	Agent/backend operational logs	<code>id</code> , <code>source</code> , <code>robot_id</code> (nullable), <code>level</code> , <code>message</code> , <code>context</code> (JSONB), <code>logged_at</code>	30 days default

Database-level tamper-resistance for `audit_logs` (new in this revision, detailed further in Part 26): the application's database role must not be granted UPDATE/DELETE on `audit_logs` at all — only INSERT and SELECT. Any correction to an audit record must be a new, linked record (`superseded_by`), never a mutation of the original.

Part 14 — APIs

Full request/response/error specification lives in `SAKAR_ROBOT_PLATFORM_API_SPEC.md`. This is a requirements specification, not an implementation — no endpoint below is built.

Representative endpoint surface:

```
POST /auth/login
POST /auth/refresh
POST /auth/logout

GET /organizations
GET /sites

GET /robots
GET /robots/{id}
GET /robots/{id}/status
GET /robots/{id}/telemetry
GET /robots/{id}/events
GET /robots/{id}/errors
GET /robots/{id}/logs
GET /robots/{id}/timeline
GET /robots/{id}/tasks
GET /robots/{id}/maps
GET /robots/{id}/history

POST /robots/{id}/commands
POST /robots/{id}/lock
POST /robots/{id}/unlock

GET /users
GET /roles
GET /permissions

GET /audit-logs
```

Every endpoint requires: authentication (bearer JWT), authorization (permission check per Part 19's matrix, scoped per Part 13's tenancy model), a defined request/response schema, and defined error responses (400/401/403/404/409/429/500-class, each with a stable error code — not just an HTTP status). `/robots/{id}/lock` and `/robots/{id}/unlock` additionally require the command-security envelope defined in Part 20, and `/robots/{id}/unlock` additionally requires step-up authentication per Part 19. None of this is implemented by this document.

Part 15 — Robot Communication

Sakar Cloud

| MQTT (telemetry/events, robot->cloud) + WebSocket (commands/live push, cloud->client) + HTTPS (request/response, both directions)

|

SakarC40Agent

| (local – CONFIRMED private-only, Part 10)

Peanut SDK

|

C40

Message types to define (fully specified in `SAKAR_ROBOT_PLATFORM_API_SPEC.md` Part "Robot Agent API"): register, heartbeat, telemetry, event, command, command acknowledgement, command result, lock, unlock, error, reconnect, offline-mode entry/exit.

Protocol comparison and recommendation (full analysis in `SAKAR_ROBOT_PLATFORM_ARCHITECTURE.md`):

Option	Real-time telemetry	Commands	Offline robots	NAT	Security	Scalability	Reconnection	Ack	Dup commands	Timeout
A: HTTPS polling	Weak (latency = poll interval)	Simple	Simple (just resumes polling)	Trivial (outbound only)	Standard TLS	Good (stateless)	Trivial	Explicit per-request	Natural (idempotency key)	Natural (request timeout)
B: WebSocket	Good	Good	Requires reconnect logic	Needs care (keep-alives through NAT)	TLS + auth on connect	Moderate (stateful connections)	Must be built	Must be built (ack messages)	Must be built	Must be built
C: MQTT	Good, purpose-built	Good, purpose-built (QoS levels)	Best — designed for intermittent devices, retained messages, LWT (last-will) for offline detection	Good (outbound persistent connection)	TLS + client-cert/token auth	Good (broker-mediated)	Native (broker handles it)	Native (QoS 1/2)	Native (QoS 1 dedup patterns)	Natural (broker + app-level)
D: Hybrid (recommended)	Good	Good	Best	Good	Good	Good	Good	Good	Good	Good

Recommendation: Option D — Hybrid. MQTT for the robot-agent-facing channel (telemetry, events, command delivery, offline detection via LWT); WebSocket for the web/mobile-facing channel (live dashboard push); plain HTTPS REST for request/response administrative operations. Security requirements for the MQTT and WebSocket legs are detailed in Part 23 and Part 24 respectively.

Part 16 — Security Architecture

Core principle (repeated from Part 6): the web and mobile applications communicate through Sakar Backend only. They must never communicate directly with a robot, under any circumstance, at any layer.

Hardened deployment view:





Defense-in-depth layers, outside-in:

Layer	Purpose	Status
WAF / Firewall	Filters malicious traffic (OWASP-class attacks, volumetric abuse) before it reaches Nginx	REQUIREMENT
Nginx	TLS termination, request routing, coarse rate limiting	REQUIREMENT
Sakar Backend	Enforces authentication, RBAC, tenancy, command security (Parts 19-20)	REQUIREMENT
Private network segmentation	PostgreSQL, Redis, MQTT broker, and monitoring have no public network exposure — reachable only from the backend and, for MQTT, from authenticated robot agents	REQUIREMENT
SakarC40Agent	Enforces local command validation before ever calling the Peanut SDK (Part 20/21)	REQUIREMENT
Peanut SDK / C40	The physical boundary — everything above exists to protect this	N/A (vendor component)

Explicit non-goal, stated as a control: neither the web application nor the mobile application is ever granted network reachability to any robot, any MQTT broker topic outside its own authorized scope, or any agent directly. This is enforced by network segmentation (this diagram) and by application-layer authorization (Part 19), redundantly — either control failing alone must not be sufficient to violate this principle.

Part 17 — Security Threat Model

#	Threat actor / scenario	Primary attack path	Primary mitigation (this document's part)	Status
T1	External attacker via the internet	Exploits a web/API vulnerability to gain unauthorized access	WAF, input validation, authentication, RBAC (Part 16/19)	REQUIREMENT
T2	Malicious or compromised customer account	Uses valid credentials to access another tenant's data or robots	Multi-tenant isolation, server-side authorization on every request (Part 19)	REQUIREMENT
T3	Insider threat (Sakar staff)	Abuses legitimate elevated access	RBAC least-privilege, audit logging, step-up auth for ROBOT_UNLOCK (Part 19/11)	REQUIREMENT
T4	Compromised robot/agent device	A rooted or physically-tampered tablet is used to forge telemetry or bypass local safety controls	Device identity, agent integrity checks, secure local storage (Part 21)	REQUIREMENT
T5	Network-level MITM (agent<->cloud or client<->cloud)	Intercepts or tampers with traffic in transit	TLS everywhere, certificate validation, no cleartext traffic (Part 22/25)	REQUIREMENT
T6	Physical device theft (robot tablet)	A stolen tablet is used to extract credentials or operate the robot	Android Keystore, secure local storage, device-owner/kiosk enforcement (Part 21)	REQUIREMENT
T7	Supply-chain / dependency compromise	A compromised third-party library introduces a vulnerability into the backend, web, mobile, or agent codebase	Dependency scanning, pinned versions, CI/CD security gates (Part 29)	REQUIREMENT

T8	Command replay	An attacker captures and re-sends a previously valid command (e.g., an old UNLOCK)	Nonce + expiration on every command (Part 20)	REQUIREMENT
T9	Command forgery	An attacker crafts a command without going through authorized channels	Command signing, agent-side local validation (Part 20)	REQUIREMENT
T10	Cross-robot message leakage (MQTT)	Robot A receives or can subscribe to Robot B's commands/telemetry	Per-robot topic isolation + broker ACLs (Part 23)	REQUIREMENT
T11	Keenon-side compromise or unexpected behavior	The stock Keenon app, Keenon Cloud, or the OTA subsystem behaves in a way Sakar did not anticipate or authorize	Documented as an accepted residual risk pending Part 22/38 network testing; architecture does not depend on Keenon Cloud for core function (Part 10)	REQUIRES NETWORK TEST / REQUIRES VENDOR SUPPORT
T12	Lock bypass via an unmanaged/uncontrolled device	Any app with SDK access calls enable(unlock) without going through Sakar's authorization	OS-level device management (kiosk/device-owner), Part 21	REQUIRES PHYSICAL C40 TEST
T13	Database compromise	Direct or exfiltrated access to PostgreSQL	Private-network-only DB, least-privilege roles, encryption at rest (Part 22/25/26)	REQUIREMENT
T14	Backup compromise	Stolen or leaked backup media exposes historical data	Encrypted backups, access-controlled backup storage (Part 27)	REQUIREMENT

This table is the narrative counterpart to the formal Part 18 register — each threat here maps to one or more numbered risks there.

Part 18 — Security Risk Register

The full standalone register (with mitigation detail and validation notes) is maintained in `SAKAR_SECURITY_RISK_REGISTER.md`. Summary below:

ID	Risk	Severity	Current Status	Mitigation	Validation
----	------	----------	----------------	------------	------------

R01	Cloud/backend compromise	Critical	REQUIREMENT	WAF, hardened Nginx/backend config, patch management, network segmentation	Penetration test (Part 29)
R02	Database compromise	Critical	REQUIREMENT	Private network only, least-privilege roles, encryption at rest	DB security review (Part 22)
R03	Credential theft (user)	High	REQUIREMENT	Password hashing, MFA, brute-force protection (Part 19)	Auth security test (Part 29)
R04	Token theft (JWT/refresh)	High	REQUIREMENT	Short-lived access tokens, refresh rotation, revocation (Part 19)	Token security test (Part 29)
R05	Robot command forgery	Critical	REQUIREMENT	Command signing, agent-side validation (Part 20)	Command security test (Part 29)
R06	Command replay	High	REQUIREMENT	Nonce + expiration (Part 20)	Replay attack test (Part 29)
R07	Cross-tenant access	Critical	REQUIREMENT	Server-side authorization on every request, tenancy hierarchy (Part 19)	IDOR/BOLA test (Part 29)
R08	MQTT compromise	High	REQUIREMENT	TLS, per-robot credentials, topic ACLs (Part 23)	MQTT security test (Part 29)
R09	WebSocket compromise	Medium	REQUIREMENT	Auth on connect, origin validation, tenant-scoped subscriptions (Part 24)	WebSocket security test (Part 29)
R10	Android device compromise	High	REQUIREMENT	Keystore, secure storage, device management (Part 21)	Android security test (Part 29)
R11	SakarC40Agent compromise	High	REQUIREMENT	App integrity, minimal permissions, secure components (Part 21)	Agent security review (Part 29)

R12	Stock Keenon app bypasses Sakar lock	High	REQUIRES PHYSICAL C40 TEST	OS-level device management; documented as unresolved until tested	Part 38, test 8
R13	Remote lock does not physically work as expected	Critical	REQUIRES PHYSICAL C40 TEST	Full Part 38 validation plan before production claim	Part 38, tests 1-3, 10
R14	Unexpected OTA/update network communication	Medium	REQUIRES NETWORK TEST	Isolated VLAN packet capture (Part 22)	Part 22/38 network test
R15	Unknown Keenon network traffic (stock app, OS services)	Medium	REQUIRES NETWORK TEST	Same as R14	Part 22/38 network test
R16	Insider threat (Sakar staff misuse)	Medium	REQUIREMENT	Least-privilege RBAC, audit logging, step-up auth for unlock	Audit review process
R17	Backup compromise	High	REQUIREMENT	Encrypted backups, access-controlled storage (Part 27)	Backup security review
R18	Dependency vulnerability	Medium	REQUIREMENT	Dependency scanning, patch cadence, CI/CD gate (Part 29)	Automated scanning

Formal risk register maintenance requirement: this register must be reviewed at every security acceptance gate (Part 35) and updated with the outcome of each physical/network test as it completes — a risk does not leave this register by being deleted, only by being re-graded with evidence attached.

Part 19 — Authentication & RBAC

RBAC model refinement note: this Part defines the authoritative role and permission model for the platform, refining the provisional set used in Part 7. Roles: SUPER_ADMIN, ORG_ADMIN, SITE_ADMIN, OPERATOR, TECHNICIAN, VIEWER. Permissions: ROBOT_VIEW, ROBOT_CONTROL, ROBOT_TASK_CREATE, ROBOT_TASK_CANCEL, ROBOT_LOCK, ROBOT_UNLOCK, ROBOT_CONFIGURE, ROBOT_DIAGNOSTICS, ROBOT_LOG_VIEW, AUDIT_VIEW, USER_MANAGE, ROLE_MANAGE, SYSTEM_ADMIN.

19.A Authentication requirements

Control	Requirement	Status
Password hashing	Modern, salted, adaptive hashing (e.g., bcrypt/Argon2-class algorithm) — never reversible encryption, never unsalted	REQUIREMENT
Password policy	Minimum length and complexity; checked against known-breached-password lists	REQUIREMENT
Brute-force protection	Progressive delay and/or lockout after repeated failed attempts, per account and per source IP	REQUIREMENT
Rate limiting	Applied at the login endpoint independently of the general API rate limit	REQUIREMENT
Account lockout	Temporary lockout after a defined failed-attempt threshold, with a defined unlock path (time-based and/or admin-assisted)	REQUIREMENT
JWT	Signed access tokens carrying user identity, org/site scope, and permission claims	REQUIREMENT
Short-lived access tokens	Minutes-scale lifetime, not hours/days	REQUIREMENT
Refresh tokens	Longer-lived, used only to mint new access tokens	REQUIREMENT
Refresh token rotation	Each refresh use issues a new refresh token and invalidates the old one; reuse of an invalidated refresh token is treated as a compromise signal	REQUIREMENT
Token revocation	Server-side capability to revoke a session immediately, independent of token expiry	REQUIREMENT
Session invalidation	Logout, password change, and role/permission change all invalidate existing sessions	REQUIREMENT
MFA	Available to all roles; required for SUPER_ADMIN/ORG_ADMIN	REQUIREMENT
Administrator MFA	Non-optional for any role holding SYSTEM_ADMIN, USER_MANAGE, or ROLE_MANAGE	REQUIREMENT
Step-up authentication for sensitive actions	Required before ROBOT_UNLOCK is exercised, regardless of whether the session's original login used MFA	REQUIREMENT

Secure password reset	Time-limited, single-use reset token delivered out-of-band; no password ever transmitted or displayed in plaintext	REQUIREMENT
Secure logout	Invalidates both access and refresh tokens server-side, not merely a client-side token discard	REQUIREMENT

These are requirements only — no implementation change is prescribed or has been made by this document.

19.B Authorization / RBAC

Role -> permission matrix (baseline; refine during design review):

Permission	SUPER_ADMIN	ORG_ADMIN	SITE_ADMIN	OPERATOR	TECHNICIAN	VIEWER
ROBOT_VIEW	Y	Y	Y	Y	Y	Y
ROBOT_CONTROL	Y	Y	Y	Y	Y	-
ROBOT_TASK_CREATE	Y	Y	Y	Y	-	-
ROBOT_TASK_CANCEL	Y	Y	Y	Y	Y	-
ROBOT_LOCK	Y	Y	Y	Y	-	-
ROBOT_UNLOCK	Y	Y (own org)	-	-	-	-
ROBOT_CONFIGURE	Y	Y (own org)	Y (own site)	-	Y	-
ROBOT_DIAGNOSTICS	Y	Y	Y	-	Y	-
ROBOT_LOG_VIEW	Y	Y	Y	Y	Y	Y
AUDIT_VIEW	Y	Y (own org)	-	-	-	-
USER_MANAGE	Y	Y (own org)	-	-	-	-
ROLE_MANAGE	Y	-	-	-	-	-

SYSTEM_ADMIN	Y	-	-	-	-	-
--------------	---	---	---	---	---	---

Rationale: ROBOT_UNLOCK is granted only to SUPER_ADMIN and ORG_ADMIN (own organization), never to SITE_ADMIN, OPERATOR, or TECHNICIAN by default — reflecting the principle that unlocking must be deliberately harder to obtain than locking. SITE_ADMIN can lock (a fail-safe action) but must escalate to an org-level admin to unlock. Adjust per real organizational policy during implementation, but preserve this asymmetry.

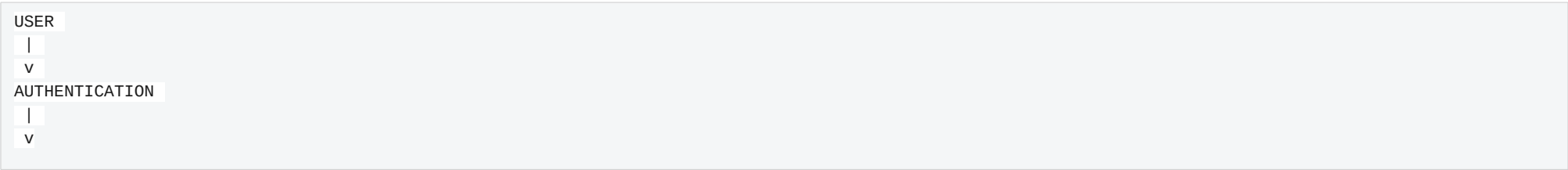
Robot authorization hierarchy: Organization -> Site -> Robot. A user's accessible-robot set is the union of robots under every site they're granted access to. **Every API call and UI list filters through this hierarchy server-side — a client-supplied robot_id is never trusted without a server-side authorization check.**

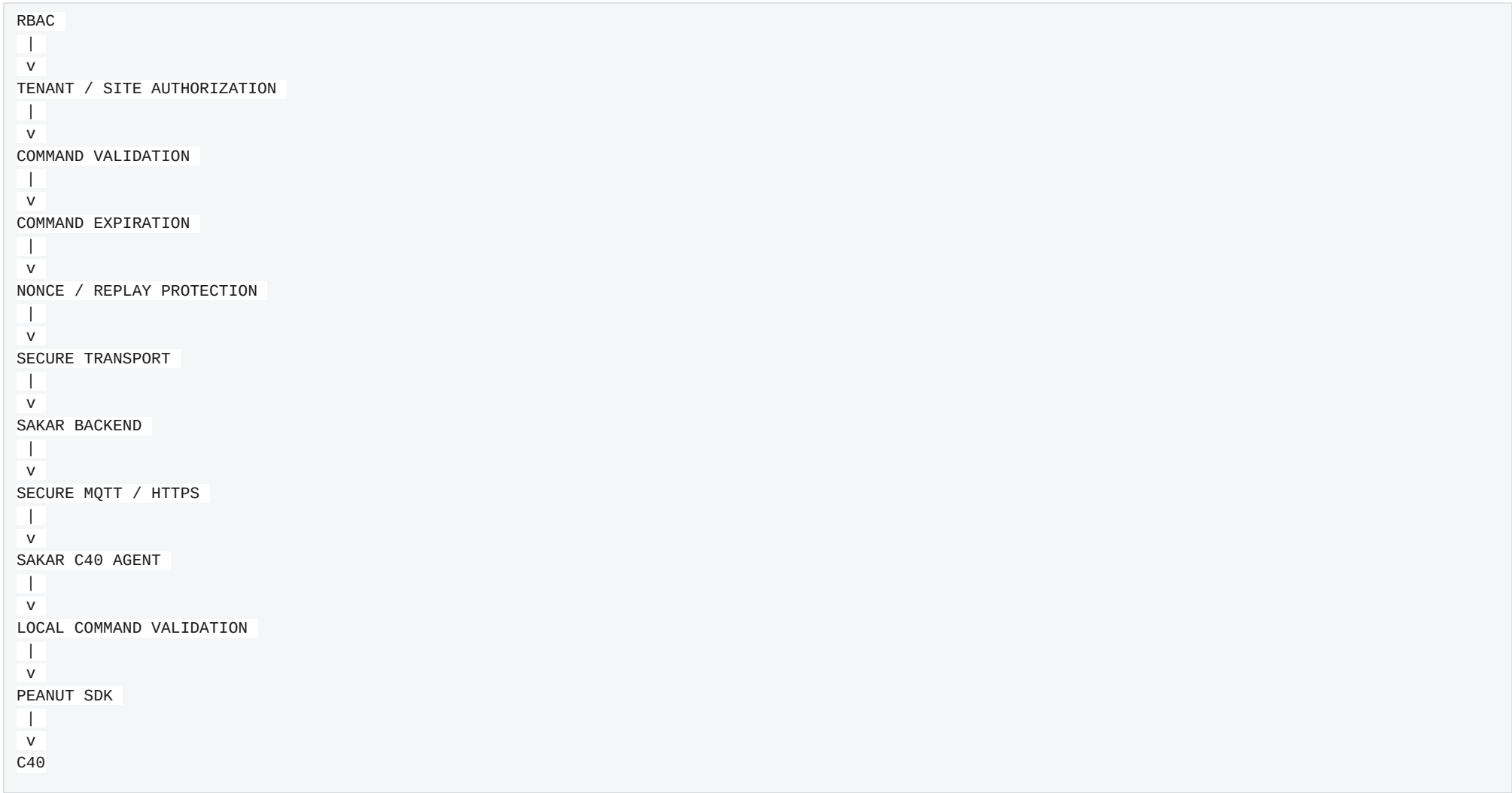
Explicit protections required against:

Vulnerability class	Requirement
IDOR (Insecure Direct Object Reference)	Every object lookup by ID must re-verify the requesting user's authorization for that specific object, not just that the ID is well-formed
BOLA (Broken Object-Level Authorization)	Same as IDOR, applied specifically to API objects (robots, tasks, commands, logs) — the API layer must not assume authorization was already checked upstream
Cross-tenant access	No query path, cache key, or WebSocket subscription may return data for an organization the requesting user is not authorized for, even transiently
Privilege escalation	A user must never be able to grant themselves or another user a role/permission outside what their own role is authorized to assign (Part 7, Users/Roles/Permissions module)

Part 20 — Robot Command Security

Secure command lifecycle:





Every command conceptually contains: `command_id`, `robot_id`, `user_id`, `timestamp`, `expiration`, `nonce`, `command_type`, `request_id`, and an authorization context (the permission and tenancy scope the issuing user held at authorization time).

Requirement	Description	Status
-------------	-------------	--------

Replay protection	A previously-used nonce for a given robot is rejected on reuse	REQUIREMENT
Command expiration	Every command carries an expiration; an agent must reject an expired command without executing it	REQUIREMENT
Duplicate command protection	Idempotent handling keyed on <code>command_id</code> — a retried delivery of the same command is a no-op, not a re-execution	REQUIREMENT
Robot identity validation	The agent verifies the command's <code>robot_id</code> matches its own identity before acting on it	REQUIREMENT
Command authorization	The backend verifies the issuing user's permission and tenancy scope at issuance time, and the command carries that authorization context for the agent to also check	REQUIREMENT
Command audit	Every command's full lifecycle (Part 12.D) is recorded, win or fail	REQUIREMENT
Command acknowledgement	The agent acknowledges receipt (<code>COMMAND_RECEIVED</code>) independently of reporting the eventual result	REQUIREMENT
Command result	The agent reports a terminal result (<code>COMMAND_SUCCESS</code> / <code>COMMAND_FAILED</code>) distinct from the acknowledgement	REQUIREMENT
Command timeout	The backend marks a command <code>COMMAND_TIMEOUT</code> if no result arrives within a defined window — never silently assumed successful	REQUIREMENT
Local command validation	The agent independently re-validates freshness/nonce/scope before invoking the Peanut SDK, even though the backend already validated — defense in depth against a compromised transport	REQUIREMENT

Part 21 — Android / SakarC40Agent Security

These are requirements only. None of the following has been implemented; SakarC40Agent's existing source code was read-only during this review and was not modified.

Control	Requirement	Status
Android Keystore	Robot credentials and any locally-held signing material are stored in the Android Keystore, never in plain files or <code>SharedPreferences</code>	REQUIREMENT
Secure local storage	Any locally-cached telemetry/command data is stored in an encrypted store appropriate to its sensitivity (Part 25 classification)	REQUIREMENT
No hardcoded secrets	No credential, key, or secret is compiled into the agent's APK — see Part 26	REQUIREMENT
Release build hardening	Production builds are signed release builds with debugging/logging disabled	REQUIREMENT
Debug disabled in production	<code>android:debuggable="false"</code> and no verbose logging of sensitive data in production builds	REQUIREMENT
Minimal permissions	The agent requests only the Android permissions it demonstrably needs — every permission currently declared (<code>INTERNET</code> , <code>ACCESS_NETWORK_STATE</code> , <code>READ_PHONE_STATE</code>) should be re-justified against this principle at implementation time	REQUIREMENT
Secure exported components	No <code>Activity/Service/Receiver/Provider</code> is exported unless it explicitly needs to be, and every exported component enforces its own permission check	REQUIREMENT
Secure services	Any background service handling telemetry/commands validates its caller	REQUIREMENT
Secure receivers	Any broadcast receiver validates the broadcast's origin before acting	REQUIREMENT
Certificate validation	TLS certificate validation is never disabled/bypassed for backend or MQTT broker connections in production builds	REQUIREMENT

TLS	All agent<->backend and agent<->broker traffic is TLS-protected	REQUIREMENT
No cleartext traffic	Android's network security config disallows cleartext HTTP for all production endpoints	REQUIREMENT
App integrity	A mechanism to detect a tampered/repackaged agent build (e.g., signature verification) is evaluated at implementation time	REQUIREMENT
Device identity	Each agent install has its own registered device identity, distinct from the robot's own identity	REQUIREMENT
Agent identity / version	The backend can identify which agent build/version is talking to it, to detect unexpected/outdated agents (Part 28)	REQUIREMENT
Local security	The agent enforces its own local authorization gate (the existing guard()/OperatingMode pattern) as defense-in-depth, independent of backend-side authorization	REQUIREMENT (extends existing pattern)
ADB restrictions	USB debugging is disabled on production robot tablets; if field debugging is ever required, it is a deliberate, audited, time-boxed exception	REQUIREMENT
USB debugging restrictions	Same as above	REQUIREMENT
Kiosk / device-owner requirements	Where Part 11/38 determines OS-level enforcement is necessary to make the lock feature meaningful , the robot tablet is enrolled in Android device-owner/kiosk mode restricting which applications (including, specifically, the stock Keenon app) may run	REQUIRES PHYSICAL C40 TEST (necessity) + REQUIREMENT (once necessity is confirmed)

Explicit instruction preserved from the review request: none of the above is implemented now. This Part documents requirements for a future implementation phase (Part 33).

Part 22 — Network Security

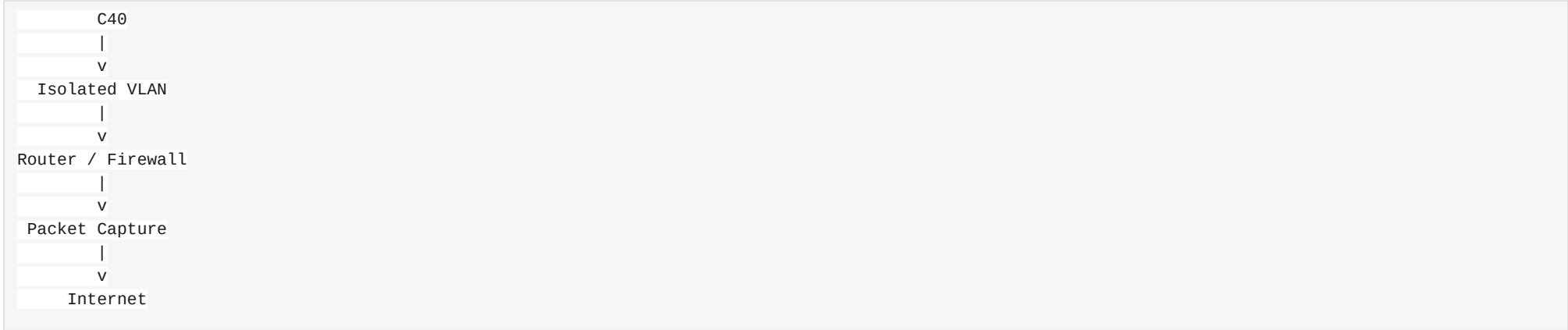
Control	Requirement	Status
Public database access	None — PostgreSQL, Redis, and the MQTT broker are reachable only from within the private network (Part 16)	REQUIREMENT
Firewall rules	Default-deny; only the specific ports/protocols each component needs are opened, and only to the specific peers that need them	REQUIREMENT
WAF	Filters common web/API attack patterns before traffic reaches the application	REQUIREMENT
DDoS/volumetric protection	Provided by the WAF/CDN layer or equivalent	REQUIREMENT
Network segmentation	Backend, database tier, and monitoring are in logically/physically separate network segments from each other and from any test/staging environment	REQUIREMENT
VPN/bastion for administrative access	Direct administrative access to production infrastructure is never exposed to the open internet	REQUIREMENT

22.A Keenon Network Security — Physical Test Requirement

This subsection documents an **unresolved** area, deliberately, rather than asserting a conclusion the source material does not support.

Item	Status
Stock Keenon application's own network traffic	UNKNOWN — REQUIRES NETWORK TEST
OTA/update subsystem traffic	UNKNOWN — REQUIRES NETWORK TEST
Unexpected outbound traffic from the robot's Android computer generally	UNKNOWN — REQUIRES NETWORK TEST
DNS requests made by the device	UNKNOWN — REQUIRES NETWORK TEST
HTTPS connections to unknown external endpoints	UNKNOWN — REQUIRES NETWORK TEST

Future physical test design (not executed by this document):



Capture and record, for every connection observed: destination IP, resolved domain (via DNS capture, not assumption), protocol, port, connection frequency, and — to the extent inferable — connection purpose (telemetry, OTA check, licensing check, etc.).

Status: REQUIRES PHYSICAL NETWORK TEST. No summary of this document may state that Keenon outbound communication has been eliminated or fully characterized until this test has been performed and its results incorporated into Part 10, Part 17 (T11), Part 18 (R14/R15), and this section.

Part 23 — MQTT Security

Control	Requirement	Status
TLS	All MQTT connections (broker<->agent, broker<->backend) are TLS-protected	REQUIREMENT
Unique robot credentials	Each robot's agent authenticates with its own unique credential — no shared/global MQTT credential	REQUIREMENT
Unique robot certificates (where feasible)	Client-certificate authentication per robot is the preferred long-term posture; evaluated for feasibility at implementation time	REQUIREMENT
ACL (Access Control List)	The broker enforces per-client topic ACLs — a robot's credential can only publish/subscribe to its own topics	REQUIREMENT

Topic isolation	Topics are namespaced by organization, site, and robot (see scheme below)	REQUIREMENT
Organization isolation	No credential from Organization A can access any topic under Organization B's namespace	REQUIREMENT
Robot isolation	Robot A must never receive Robot B's commands — enforced by the ACL matching the topic scheme exactly, not by application-layer filtering alone	REQUIREMENT
Credential rotation	Robot MQTT credentials are rotatable without requiring a full agent redeployment	REQUIREMENT
Connection limits	The broker enforces per-credential connection limits to contain a compromised or malfunctioning client	REQUIREMENT
Message validation	The backend validates every inbound MQTT message's schema and sender before acting on it — the broker delivering a message is not itself proof of validity	REQUIREMENT

Topic scheme:

```
sakar/{organization}/{site}/{robot}/telemetry
sakar/{organization}/{site}/{robot}/events
sakar/{organization}/{site}/{robot}/commands
sakar/{organization}/{site}/{robot}/ack
```

Each robot's credential is scoped by ACL to only its own four topics (publish on telemetry/events/ack, subscribe on commands). This is the mechanism, combined with per-robot credentials, that makes "Robot A must never receive Robot B's commands" an enforceable property rather than a hope.

Part 24 — WebSocket Security

Control	Requirement	Status
---------	-------------	--------

Authentication	A WebSocket connection is only accepted from a client presenting a valid, unexpired access token	REQUIREMENT
Authorization	Every subscription request is checked against the user's robot-level authorization (Part 19) at subscribe time, not only at connect time	REQUIREMENT
Tenant isolation	A user may only receive events for robots under organizations/sites they are authorized for — enforced server-side per message, not just per connection	REQUIREMENT
Origin validation	The server validates the Origin header against an allow-list of known Sakar web/mobile origins	REQUIREMENT
Rate limiting	Both connection-establishment rate and message rate are limited per client	REQUIREMENT
Connection limits	A cap on concurrent WebSocket connections per user/session, to contain abuse	REQUIREMENT
Token expiry	A WebSocket connection tied to an access token is terminated (or forced to re-authenticate) when that token expires — a long-lived socket must not outlive a short-lived token	REQUIREMENT
Message validation	Every inbound message is schema-validated before processing	REQUIREMENT
Heartbeat	Application-level ping/pong to detect and clean up dead connections	REQUIREMENT
Timeout	Idle connections are closed after a defined inactivity window	REQUIREMENT

Part 25 — Data Protection

Data classification:

Classification	Examples from this platform's data model
PUBLIC	Marketing material, public documentation
INTERNAL	Aggregate/anonymized analytics, non-sensitive configuration
CONFIDENTIAL	Robot telemetry, robot events/errors, task/cleaning history, site information, customer organizational information
HIGHLY_CONFIDENTIAL	Robot credentials, user credentials, access/refresh tokens, MFA secrets, audit records, any signing/API key

Requirements:

Control	Requirement	Status
Transport encryption	TLS 1.2 minimum, TLS 1.3 preferred, for every network hop with no exceptions (including internal traffic between backend and database/cache/broker)	REQUIREMENT
Encryption at rest	Database volume encryption; encrypted backups (Part 27)	REQUIREMENT
Secret handling in logs	Secrets, tokens, and credentials are never written to any log (application, access, or audit) — logging code must explicitly redact known-sensitive fields	REQUIREMENT
Classification-driven handling	HIGHLY_CONFIDENTIAL data has stricter access, retention, and export controls than CONFIDENTIAL data — enforced by the RBAC permissions that gate each (e.g., AUDIT_VIEW vs ROBOT_VIEW)	REQUIREMENT
Data minimization	Only data with a defined product/operational purpose (Part 9) is collected — no speculative data collection	REQUIREMENT

Part 26 — Secrets Management

No production secret may ever appear in: React source, mobile app source/bundles, Git history (including old commits), Dockerfiles, public configuration files, or APK resources (strings, assets, or build config).

Control	Requirement	Status
No hardcoded passwords	Never in source, config committed to Git, or container images	REQUIREMENT
No hardcoded API keys	Same	REQUIREMENT
No hardcoded JWT signing secrets	Same	REQUIREMENT
No hardcoded database passwords	Same	REQUIREMENT
No MQTT passwords in source code	Robot/broker credentials are provisioned at runtime, never compiled in	REQUIREMENT
No private keys in Git	Including historical commits — a leaked key found in history must be treated as compromised even if later removed	REQUIREMENT
Secret rotation	All secrets (API keys, signing keys, database credentials) have a defined rotation procedure and cadence	REQUIREMENT
Credential rotation	Robot/agent credentials are individually rotatable (Part 23)	REQUIREMENT
Environment separation	Development, staging, and production use entirely distinct secrets — a staging credential must never grant production access	REQUIREMENT
Secret manager	A dedicated secrets store (not environment variables baked into images, not a shared config file) holds all production secrets	REQUIREMENT
Secure robot credentials	Each robot's MQTT/backend credential is unique, provisioned at registration time, and stored per Part 21's Android Keystore requirement on the device side	REQUIREMENT

Self-check applied to this document itself: no credential, token, or secret value from any prior audit (KEENON_C40_CLOUD_API_AUDIT.md, KEENON_C40_API_TEST_RESULTS.json, or any Postman session) appears anywhere in this document or its companions — this was verified by direct text search before publication (Part 39/Final Report).

Part 27 — Backup & Disaster Recovery

Control	Requirement	Status
Backup frequency	Daily, at minimum, for the production database	REQUIREMENT
Backup encryption	Every backup is encrypted at rest, using keys managed separately from the backup storage itself	REQUIREMENT
Separate backup storage	Backups are stored in a location/account distinct from the primary production environment, so a compromise of production does not automatically compromise backups	REQUIREMENT
Restore testing	Restore procedures are tested on a defined cadence (not just written and assumed to work)	REQUIREMENT
RPO (Recovery Point Objective)	Defined at architecture sign-off; a daily-backup baseline implies an RPO on the order of 24 hours unless a tighter target is chosen	REQUIREMENT
RTO (Recovery Time Objective)	Defined at architecture sign-off	REQUIREMENT
Disaster recovery plan	A documented runbook exists for restoring service after a major infrastructure loss, not only for restoring a single corrupted table	REQUIREMENT
Backup monitoring	Backup job success/failure is monitored and alerted on (Part 28) — a silently-failing backup is treated as a production incident	REQUIREMENT

Part 28 — Monitoring

Security-relevant conditions to monitor:

Condition	Why it matters
Failed logins (repeated, per account or per source)	Early signal of credential-stuffing/brute-force attempts
Repeated unlock attempts	Direct signal against the platform's highest-risk feature
Suspicious commands (unusual type/frequency/target for a given user)	Possible compromised account or insider misuse
Command replay attempts	Direct signal of an attempted replay attack (Part 20)
MQTT authentication failures	Possible credential compromise or misconfigured/rogue device
Unusual API traffic (volume, pattern, source)	Possible scraping, enumeration, or attack reconnaissance
Unusual robot activity (state transitions inconsistent with any issued command)	Possible bypass of Sakar's own command path (Part 11 T12)
Privilege changes	Every role/permission grant is a security-relevant event, always alerted on, not just logged
New admin users	SUPER_ADMIN/ORG_ADMIN creation is always alerted on
Credential changes	Password/MFA changes, especially for privileged accounts
Unexpected agent versions	An agent build the backend does not recognize connecting as if it were legitimate
Unexpected network traffic	Any traffic pattern inconsistent with the architecture in Part 16 (e.g., a robot-tablet-originated connection to an address outside the expected set)

Recommended tooling: Prometheus + Grafana for metrics/dashboards; centralized log aggregation for the six SRELS categories (Part 12) plus infrastructure logs; a dedicated security-alerting channel distinct from general operational alerting, so security signals are not lost in routine noise.

Part 29 — Security Testing

Future testing requirements (none executed by this document; no destructive testing is ever performed against the physical C40):

Test area	Scope
-----------	-------

Authentication	Password policy enforcement, brute-force protection, MFA bypass attempts, session fixation
Authorization / RBAC	Every permission in Part 19's matrix, tested against every role, including negative tests (a role must NOT be able to do X)
Tenant isolation	Cross-organization/cross-site access attempts via every API surface
IDOR	Direct object reference manipulation across every ID-bearing endpoint
BOLA	Object-level authorization bypass attempts specific to the robot/task/command API surface
Privilege escalation	Attempts to self-grant or grant-beyond-scope roles/permissions
JWT	Signature validation, algorithm confusion, expired/malformed token handling
Refresh tokens	Rotation correctness, reuse detection, revocation propagation
Replay attacks	Command and authentication replay across MQTT, WebSocket, and REST
Command forgery	Attempts to issue a command without a valid authorization chain
MQTT	ACL bypass attempts, topic enumeration, credential misuse
WebSocket	Origin bypass, cross-tenant subscription attempts, token-expiry handling
SQL injection	Every user-input-accepting endpoint
XSS	Every web-rendered user-controlled field
CSRF	Every state-changing web endpoint
SSRF	Any endpoint that fetches a URL on the server's behalf (e.g., map/image handling)
Path traversal	Any file-path-accepting endpoint (map upload/download, log export)
Rate limiting	Verification that limits are actually enforced, not just configured
Android security	Static/dynamic analysis of the agent APK per Part 21's controls
Robot agent	Local command validation bypass attempts (software-level only — never against the physical robot)

Lock/unlock	Full authorization-chain testing in software; physical effect testing is governed exclusively by Part 38, not by this general security-testing program
-------------	--

CI/CD security gate (Part 33): dependency vulnerability scanning and, at minimum, static analysis for the categories above are run as part of the build pipeline before any release candidate is promoted toward production.

Part 30 — Non-Functional Requirements

Category	Requirement
Performance	API p95 < 500ms for read endpoints under nominal load
Availability	Backend core services target 99.5%+ monthly; robot local safety must never depend on backend availability
Scalability	Telemetry/Command path independently scalable from Auth/User path
Reliability	At-least-once command delivery with idempotency; bounded, documented telemetry-loss window during outages
Security	Per Parts 16-29
Observability	Structured logs, end-to-end request tracing via <code>request_id</code> , metrics on command latency and telemetry freshness
Backup	Per Part 27
Disaster recovery	Per Part 27
Data retention	Per Part 13's per-table policy
Offline support	Per the offline/failure matrix in <code>SAKAR_ROBOT_PLATFORM_REQUIREMENTS.md</code>
Command latency	Target a few seconds end-to-end on a healthy network; must be measured once the transport (Part 15) is implemented

Telemetry frequency	Configurable; default interval set during Phase 0/1 based on real link latency (UNKNOWN until the C40's actual LinkType is confirmed)
Logging performance	application_logs ingestion must never block telemetry ingestion — separate write paths
Time to detect (security)	A defined target for how quickly a Part 28 monitored condition surfaces as an alert, set at implementation time
Time to respond (security)	A defined target for incident-response initiation after an alert (Part 35 gates the go-live decision on this being defined, not necessarily minimized to zero before launch)

Part 31 — UI Requirements

Web (21 screens): Login | Dashboard | Organizations | Sites | Robots | Robot Details | Live Monitoring | Maps | Tasks | Cleaning | Logs | Events | Errors | Alerts | Analytics | Users | Roles | Permissions | Audit Logs | Robot Lock Management | Settings.

Mobile (10 screens): Login | Dashboard | Robot List | Robot Details | Live Monitoring | Tasks | Alerts | Logs | Lock/Unlock | Profile.

Robot Tablet (8 screens): Boot | Agent Status | Robot Status | Connection | Lock Status | Diagnostics | Error | Maintenance.

Security-driven UI requirements (new in this revision): the Lock/Unlock screens (web, mobile) must visibly render the maturity flag from Part 11 (Planned / Available / Requires Test) — this is not optional chrome, it is how the product avoids overclaiming a safety guarantee. The Audit Logs screen must have no delete or edit affordance of any kind, consistent with Part 13's append-only requirement.

Part 32 — Tech Stack

Layer	Technology	Role
Robot	Android + Kotlin/Java + Peanut SDK	Existing platform; SakarC40Agent is already built this way
Backend	Java + Spring Boot + Spring Security	Application logic, REST API, MQTT/WebSocket integration, AuthN/AuthZ

Database	PostgreSQL	System of record (Part 13)
Cache	Redis	Sessions, hot-path telemetry reads, rate limiting
Robot communication	MQTT	Agent-facing real-time transport (Part 15/23)
Web real-time	WebSocket	Dashboard live push (Part 15/24)
Web	React + TypeScript	Type-safe, component-based, the dominant ecosystem choice for a data-dense admin dashboard
Mobile	Flutter	Single codebase for Android + iOS
Infrastructure	Docker + Linux + Nginx	Consistent deployment, TLS termination, reverse proxy
Monitoring	Prometheus + Grafana	Operational metrics (Part 28)
Logging	Sakar Robot Log/Event System (SRELS)	The product's own domain-specific logging capability (Part 12), not a generic log aggregator

No change from v1.0 — this security review found no basis in the source material to change the recommended stack; every new security requirement (Parts 16-29) is achievable within it.

Part 33 — Development Roadmap

Phase	Objective	Deliverables	Dependencies	Risks	Acceptance criteria
0	Physical C40 SDK validation	Resolved <code>LinkType</code> ; confirmed/refuted lock physical effect (Part 38); confirmed/refuted position data; confirmed/refuted lock persistence; stock-app interaction result	Physical C40 access	Robot unavailable, SDK behaves unexpectedly	Every REQUIRES PHYSICAL C40 TEST row in this document resolved

0.5	Physical Keenon network validation	Packet-capture results for Part 22.A	Isolated VLAN test setup, physical C40 access	Test environment doesn't reflect production network conditions	Part 22.A table fully populated with real findings
1	SakarC40Agent extension	Telemetry-forwarding capability added; agent-side command reception scaffold; Part 21 controls designed	Phase 0 findings	Extending existing code without breaking current diagnostic functionality	Agent forwards a defined telemetry set to a test backend reliably
2	Sakar Cloud Backend	Core services: Auth, Robot Registry, Telemetry, Command; Part 19/20 controls designed	Phase 1	Over-scoping into premature microservices	Backend ingests telemetry; issues a signed command the agent can validate
3	Database	Full schema per Part 13 deployed; Part 26 secrets-management posture in place	Phase 2	Schema churn if built before requirements stabilize	All Part 13 tables exist with defined indexes/retention
4	SRELS	SRELS tables + ingestion (Part 12)	Phase 3	Conflating log types back into one table under time pressure	Timeline (Part 12.F) renders correctly across all six log categories
5	Security Foundation	Parts 19, 21, 22, 25, 26 implemented at the infrastructure/backend level	Phases 2-4	Treating security as a bolt-on after features are "done"	No P0 feature ships without its security controls in place
6	Web Dashboard	Modules per Part 7.A	Phases 2-5	Feature creep beyond confirmed capabilities	Every module's acceptance criteria pass against real/simulated data
7	Mobile App	Modules per Part 7.B	Phase 6 (shares backend APIs)	Divergent permission model from web	Mobile enforces identical RBAC to web
8	Remote Commands	Navigation/task/cleaning command paths (non-lock), Part 20 controls live	Phase 5 command infrastructure + Phase 0 physical confirmation	Commands with unconfirmed physical effect shipped as if certain	Each command type carries the same maturity-flag discipline as lock
9	Remote Lock/Unlock	Production-certified lock/unlock, gated on Phase 0/38's ten-condition test	Phase 0/38, Phase 8 command infrastructure	Shipping before physical certification — explicitly disallowed	All ten Part 11/38 conditions CONFIRMED; Part 35 Gate passed

10	Security Hardening & Testing	Full Part 29 testing program executed; MQTT/WebSocket security (Parts 23/24) validated	Phases 5-9	Retrofitting security after feature-complete is harder/riskier	No critical/high findings open at sign-off (Part 35)
11	Fleet Management	Multi-org/multi-site scale-out, robot-model abstraction exercised	Phases 5-10	Data model assumptions from a single-robot-model MVP not holding	Onboarding a second robot model requires no schema/security redesign
12	Production Deployment	Go-live, monitoring (Part 28), on-call runbook, DR drill (Part 27)	All prior phases + all Part 35 gates passed	Operational readiness gaps	Defined RPO/RTO met in a production drill; every Part 35 gate green

Part 34 — Acceptance Criteria

Robot registration: *Given* an authorized admin, *when* a valid robot is registered, *then* it appears in the fleet scoped to the correct organization/site.

Telemetry: *Given* an online C40 with a functioning agent, *when* telemetry is available, *then* Sakar Backend receives and durably stores it within the ingestion window.

Dashboard: *Given* a logged-in user, *when* the Dashboard loads, *then* every count/figure reflects only robots the user is authorized for.

Mobile: *Given* a mobile session, *when* a push-notification-triggering event occurs, *then* the notification is delivered within the configured latency.

Robot Agent: *Given* SakarC40Agent running against a connected C40, *when* the agent initializes, *then* it reaches a `CONNECTED` state and begins forwarding telemetry, or surfaces an explicit, non-fabricated error state.

Commands: *Given* an authorized command, *when* it is sent, *then* its full lifecycle is recorded in `robot_commands`, and no command is silently dropped without a terminal status.

Lock: *Given* an authorized admin, *when* LOCK is requested, *then* the full software chain (authorize -> send -> receive -> invoke `MotorComponent.enable(MOTOR_ENABLE_LOCK)` -> audit log) completes and is verifiable in software — **the additional criterion "the physical C40 remains immobile" cannot be marked PASS until Part 38 is completed.**

Unlock: Same chain, requiring `ROBOT_UNLOCK` + step-up authentication; reported motor status transitions away from 255; physical resumption of normal operation is likewise gated on Part 38.

Logs: *Given* any event/error/command defined in Part 12, *when* it occurs, *then* it is queryable in the appropriate table/UI within the ingestion latency window.

Audit: *Given* any sensitive action (Part 12.E), *when* it occurs, *then* an immutable, append-only audit record is created with all required fields.

Alerts: *Given* a telemetry value crossing a configured threshold, *when* ingested, *then* an alert is created and visible within the configured latency.

Offline behavior: Per the full scenario matrix in `SAKAR_ROBOT_PLATFORM_REQUIREMENTS.md` — every scenario has a defined expected behavior, user-visible status, recovery mechanism, and safety behavior.

Security: *Given* a user authorized only for Organization A, *when* they attempt to access any resource belonging to Organization B via any endpoint, *then* the request is rejected server-side regardless of a valid session token.

Part 35 — Security Acceptance Gates

No feature governed by a gate below may be represented as production-ready until its gate shows all listed conditions met. Gates are cumulative — a later gate assumes earlier gates are green.

Gate	Covers	Conditions	Status
G1 — Authentication Hardening	Part 19.A	Password policy, brute-force protection, MFA, token rotation/revocation all implemented and tested	REQUIREMENT
G2 — Authorization & Tenant Isolation	Part 19.B	RBAC matrix enforced server-side on every endpoint; IDOR/BOLA/cross-tenant tests (Part 29) pass with zero findings	REQUIREMENT
G3 — Command Security	Part 20	Replay, expiration, duplicate-protection, and identity-validation controls implemented and tested	REQUIREMENT
G4 — Transport & Broker Security	Parts 22-24	Network segmentation live; MQTT ACLs enforced; WebSocket tenant-scoping enforced	REQUIREMENT
G5 — Secrets & Data Protection	Parts 25-26	No secret in source/Git/images; encryption at rest and in transit verified; data classification applied	REQUIREMENT

G6 — Backup & DR	Part 27	At least one successful restore drill completed against a realistic dataset	REQUIREMENT
G7 — Security Testing Sign-off	Part 29	Full testing program executed; no open critical/high findings	REQUIREMENT
G8 — Lock/Unlock Physical Certification	Parts 11, 38	All ten physical conditions CONFIRMED; stock-app bypass either disproven or mitigated by confirmed OS-level enforcement	REQUIRES PHYSICAL C40 TEST
G9 — Keenon Network Disclosure	Parts 10, 22.A	Physical packet-capture test executed; findings documented and accepted (or acted upon) by Sakar leadership before any "data stays on Sakar infrastructure" claim is made to a customer	REQUIRES PHYSICAL NETWORK TEST

Gate governance: each gate must be signed off by a named owner (Security + Engineering at minimum) before the corresponding feature set is promoted beyond staging. A gate may not be waived silently — if a launch date requires shipping ahead of a gate, that decision and its residual risk must be documented in Part 18/36 with an accountable owner, not implied by omission.

Part 36 — Risks

Risk	Status
C40 physical SDK compatibility (which LinkType)	UNKNOWN — Phase 0
Stock Keenon app interference with lock	LIKELY, unconfirmed — Part 38
SDK vendor dependency (closed-source .aar, no source access)	CONFIRMED constraint — mitigated by the existing single-chokepoint (PeanutSdkBridge) isolation
32-bit-only native library (libkeenon_serial.so ships only armeabi/armeabi-v7a)	CONFIRMED — a hard compatibility constraint on future Android/tablet OS choices
Robot firmware changes altering documented behavior	LIKELY risk over time — re-run Phase 0 tests after any firmware/SDK update
Device compromise (rooted/tampered tablet)	UNKNOWN current exposure — Part 21 is a P1 roadmap item until implemented

Security-specific risks	Tracked formally in Part 18 / SAKAR_SECURITY_RISK_REGISTER.md, not duplicated here
-------------------------	--

Part 37 — Open Questions

Question	Status
Which LinkType does the physical C40 actually require?	UNKNOWN — Phase 0
Does motor lock physically prevent movement?	REQUIRES PHYSICAL C40 TEST — Part 38
Can navigation or manual control bypass the lock?	REQUIRES PHYSICAL C40 TEST — Part 38
Does the lock persist across agent restart / robot reboot / network loss?	REQUIRES PHYSICAL C40 TEST — Part 38
Can the stock Keenon application bypass the lock?	REQUIRES PHYSICAL C40 TEST — Part 38
What Android permissions/APIs are needed for kiosk/device-owner enforcement on the specific tablet model used?	UNKNOWN — requires a device-management design spike
What does the OTA/update subsystem actually communicate with, and where?	UNKNOWN — REQUIRES NETWORK TEST
What does the stock Keenon application communicate with, and where?	UNKNOWN — REQUIRES NETWORK TEST, possibly REQUIRES VENDOR SUPPORT
Is there any other background Android-level network traffic on the tablet?	UNKNOWN — REQUIRES NETWORK TEST
What is Keenon's own position on Sakar deploying OS-level device management on their hardware?	REQUIRES VENDOR SUPPORT

Every item above must be re-reviewed at the end of Phase 0/0.5 and either resolved or explicitly carried forward with an owner and a re-test date — none may simply be dropped from tracking.

Part 38 — Physical C40 Validation Plan

General precaution for every test below: perform only in a clear, controlled area with the robot's physical emergency-stop button within immediate reach of the operator at all times. A second operator is present for any test in the Lock/Unlock or Manual Control groups specifically to trigger the physical e-stop if needed. No test in this plan is executed by this document — it is a plan for a future, explicitly authorized session.

38.A Foundational SDK/Agent Tests

#	Objective	API	Expected result	Safety requirement
1	SDK initialization	<code>PeanutSDK.getInstance().init()</code>	<code>.onInit</code> fires with success code	None
2	Runtime connection	<code>PeanutRuntime.getInstance().startHeartbeat()</code>	Heartbeat callbacks begin firing	None
3	Heartbeat content	Subscribe <code>RUNTIME_HEARTBEAT</code>	Payload matches documented shape, motor status present	None
4	Robot IP query	<code>RuntimeInfo.getRobotIp()</code>	A plausible local IP string	None
5	Battery read	<code>BatteryComponent.getStatus()</code>	Sane 0-100 percentage	None
6	Motor status read	<code>MotorComponent.getStatus()</code>	A documented status code (16/32/49/50/51/52/255)	None — read-only
7	Health read	<code>MotorComponent.getHealth()</code>	Returns without error	None
8	Emergency-state read	<code>RuntimeInfo.isEmergencyEnabled()</code> / <code>IsEmergencyEnabled</code> button	IsEmergencyEnabled state	E-stop reachable
9	Position query	<code>RuntimeComponent.getRobotPosition()</code>	Coordinates or a clean documented empty response	Robot stationary, area clear
10	Navigation status read	<code>NavigationComponent.getStatus()</code>	A documented nav-state code	Robot stationary

38.B Lock/Unlock Physical Validation (the ten mandatory conditions from Part 11)

#	Test	Objective	Safety requirement	Success criteria	Failure criteria
---	------	-----------	--------------------	------------------	------------------

1	Lock prevents physical movement	Confirm the robot cannot be driven after <code>enable(MOTOR_ENABLE_LOCK)</code> wheel-chock if possible	E-stop within reach; second operator; area clear;	No physical movement occurs when a drive command is issued while locked	Any physical movement — stop testing immediately if this occurs
2	Navigation cannot bypass lock	Attempt <code>NavigationComponent.setTarget(...)</code> while locked	Same as test 1	Command is rejected or has no physical effect	Robot moves under navigation command while locked
3	Manual control cannot bypass lock	Attempt <code>MotorComponent.manual()/forward()</code> while locked	Same as test 1	Command is rejected or has no physical effect	Robot moves under manual command while locked
4	Lock survives agent restart	Lock, force-stop and relaunch <code>SakarC40Agent</code> , re-query status	E-stop reachable	<code>getStatus()</code> still reports 255 after relaunch	Status reverts to unlocked without an explicit unlock call
5	Lock survives robot reboot	Lock, power-cycle the robot, reconnect, re-query status	E-stop reachable; only attempt after test 4 passes	Status still 255 after reboot	Status reverts without an explicit unlock call
6	Lock survives network loss	Lock, disconnect the SDK's local link, wait, reconnect, re-query	E-stop reachable	Status still 255 after reconnecting	Status reverts, implying the lock does not persist through a link drop
7	Unauthorized local user cannot bypass	Attempt to unlock/drive using any locally-available means without going through Sakar's authorization chain	E-stop reachable	No bypass path succeeds	Any successful bypass
8	Stock Keenon app cannot bypass lock	With Sakar's lock engaged, launch the stock Keenon app (if installed) and attempt to unlock/drive from it	E-stop reachable; second operator; fully controlled area; run only after tests 1-6 pass	Stock app is blocked/errors when attempting to unlock — a positive finding worth documenting in detail	Stock app successfully unlocks/drives — confirms the need for OS-level enforcement (Part 21)
9	Authorized Sakar user can unlock	<code>enable(MOTOR_ENABLE_UNLOCK)</code> after tests 1-8, confirm status and physical resumption	E-stop reachable	<code>enable()</code> succeeds, <code>getStatus()</code> moves off 255, robot can subsequently move normally	<code>enable()</code> errors, or status remains 255

10	Power-cycle behavior is understood	Document the lock state through a full power-off/power-on cycle from cold	E-stop reachable	Behavior is observed and documented, whatever it is	N/A — this test's purpose is documentation, not a pass/fail outcome, but the finding must be recorded in Part 11/18
----	------------------------------------	---	------------------	---	---

38.C Keenon Network Physical Test (Part 22.A, restated for completeness)

Isolated VLAN -> router/firewall -> packet capture -> internet, capturing destination IP, resolved domain, protocol, port, frequency, and inferred purpose for every connection observed from the robot's Android computer, run for a period long enough to observe at least one full charge/idle/task cycle and, if feasible, one OTA-check interval.

Status of this entire Part: no test has been executed. This is a plan, pending physical C40 and network-test access, explicit authorization, and a second operator's availability.

Part 39 — Final Recommendation

Build the platform in the order Part 33 specifies, starting with **Phase 0 — physical C40 validation** and **Phase 0.5 — physical Keenon network validation** — before writing a single line of production code for any feature this document marks `REQUIRES PHYSICAL C40 TEST` or `REQUIRES NETWORK TEST`. The architecture, data model, and API surface specified here are sound to begin building against today; security controls (Parts 16-29) should be built alongside features from Phase 5 onward, not retrofitted after the fact, per Part 35's gate structure.

The remote lock feature — the platform's headline safety capability — must not be represented to any customer, internal stakeholder, or marketing material as production-ready until all ten conditions in Part 11/38 are physically confirmed, and Gate G8 (Part 35) is signed off. **No claim that Keenon Cloud communication has been eliminated may be made until Gate G9 (Part 35) is signed off.** The commercial and safety cost of overclaiming either of these is asymmetric: a delayed feature is a schedule problem; a lock that doesn't actually lock, or a data-residency claim that turns out to be false, is a safety incident and a trust incident respectively. Build the honesty into the product from day one — visible maturity flags, gates that block launch rather than get waived quietly — rather than retrofitting it after a customer or an auditor discovers the gap.

Next phase: Security Requirements Review and Approval — this document, `SAKAR_SECURITY_REQUIREMENTS.md`, and `SAKAR_SECURITY_RISK_REGISTER.md` are ready for that review. No implementation should begin until that review is complete.